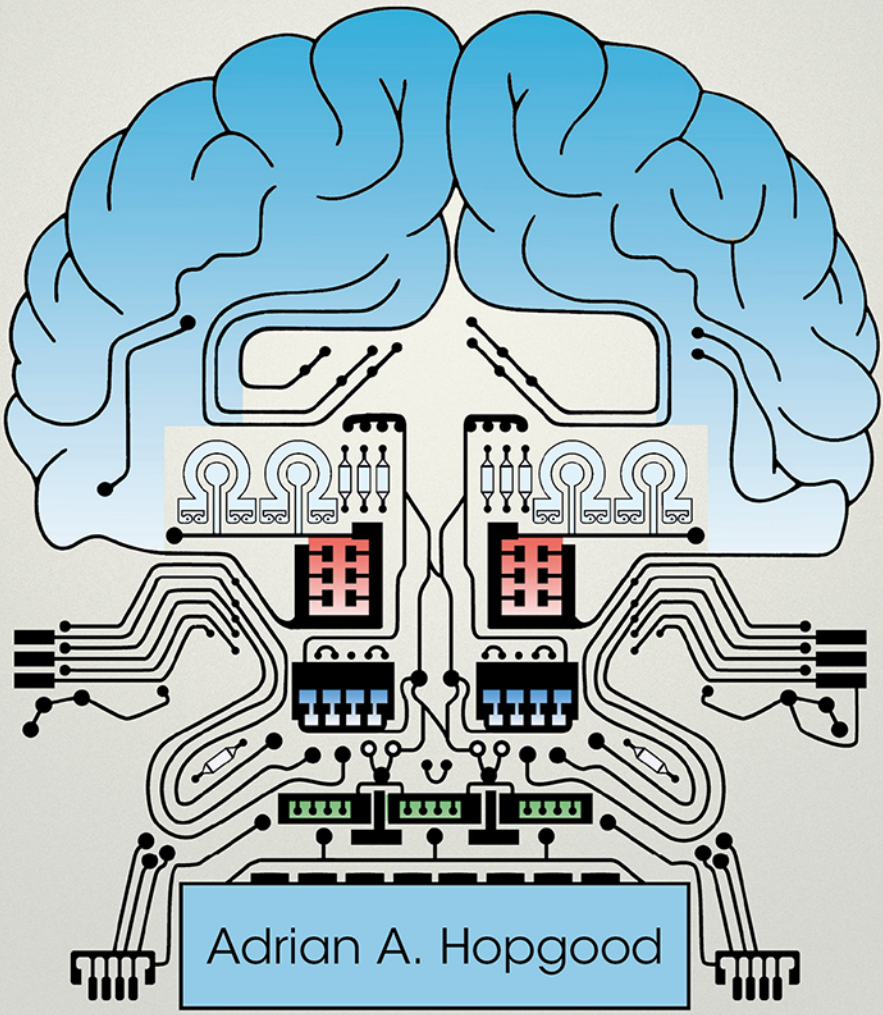
Intelligent Systems for Engineers and Scientists

A Practical Guide to Artificial Intelligence

FOURTH EDITION



CRC Press
Taylor & Francis Group
AN AUERBACH BOOK

Intelligent Systems for Engineers and Scientists

A Practical Guide to Artificial Intelligence


Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Intelligent Systems for Engineers and Scientists

A Practical Guide to Artificial Intelligence

Fourth edition

Adrian A. Hopgood



CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

Cover image provided by The Open University and used with its permission.
Flex, Flint, and VisiRule are trademarks of Logic programming Associates Ltd. http://lpa.co.uk

Fourth edition published 2022
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press
2 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

© 2022 Taylor & Francis Group, LLC

First edition published by CRC Press 1993
CRC Press is an imprint of Taylor & Francis Group, LLC

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

ISBN: 978-0-367-33616-5 (hbk)
ISBN: 978-1-032-12676-0 (pbk)
ISBN: 978-1-003-22627-7 (ebk)

DOI: 10.1201/9781003226277

Typeset in Garamond
by SPi Technologies India Pvt Ltd (Straive)

For my family


Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Contents

Preface ..... **xix**
Acknowledgements ..... **xxi**
Author ..... **xxiii**

1 Introduction **1**
1.1 Artificial Intelligence and Intelligent Systems 1
1.2 A Spectrum of Intelligent Behavior 3
1.3 Knowledge-Based Systems (KBSs) 4
1.4 The Knowledge Base 6
1.4.1 Rules and Facts 6
1.4.2 Inference Networks 7
1.4.3 Semantic Networks 8
1.5 Deduction, Abduction, and Induction 10
1.6 The Inference Engine 11
1.7 Declarative and Procedural Programming 12
1.8 Expert Systems 14
1.9 Knowledge Acquisition 15
1.10 Search 15
1.11 Computational Intelligence (CI) 17
1.12 Integration with Other Software 19
Further Reading 19

2 Rule-Based Systems **21**
2.1 Rules and Facts 21
2.2 A Rule-Based System for Boiler Control 22
2.3 Rule Examination and Rule Firing 24
2.4 Maintaining Consistency 25
2.5 The Closed-World Assumption 27
2.6 Use of Local Variables within Rules 28
2.7 Forward Chaining (a Data-Driven Strategy) 30
2.7.1 Single and Multiple Instantiation of Local Variables 31
2.7.2 Rete Algorithm 33

2.8 Conflict Resolution....35
2.8.1 First Come, First Served....35
2.8.2 Priority Values....36
2.8.3 Metarules....37
2.9 Backward Chaining (a Goal-Driven Strategy)....38
2.9.1 The Backward-Chaining Mechanism....38
2.9.2 Implementation of Backward Chaining....39
2.9.3 Variations of Backward Chaining....41
2.9.4 Format of Backward-Chaining Rules....43
2.10 A Hybrid Strategy....44
2.11 Explanation Facilities....48
2.12 Summary....49
Further Reading....49

3 Handling Uncertainty: Probability and Fuzzy Logic.....51
3.1 Sources of Uncertainty....51
3.2 Bayesian Updating....52
3.2.1 Representing Uncertainty by Probability....52
3.2.2 Direct Application of Bayes' Theorem....53
3.2.3 Likelihood Ratios....55
3.2.4 Using the Likelihood Ratios....56
3.2.5 Dealing with Uncertain Evidence....58
3.2.6 Combining Evidence....59
3.2.7 Combining Bayesian Rules with Production Rules....62
3.2.8 A Worked Example of Bayesian Updating....63
3.2.9 Discussion of the Worked Example....65
3.2.10 Advantages and Disadvantages of Bayesian Updating....66
3.3 Certainty Theory....68
3.3.1 Introduction....68
3.3.2 Making Uncertain Hypotheses....68
3.3.3 Logical Combinations of Evidence....70
3.3.3.1 Conjunction....71
3.3.3.2 Disjunction....71
3.3.3.3 Negation....71
3.3.4 A Worked Example of Certainty Theory....72
3.3.5 Discussion of the Worked Example....73
3.3.6 Relating Certainty Factors to Probabilities....74
3.4 Fuzzy Logic: Type-1....75
3.4.1 Crisp Sets and Fuzzy Sets....76
3.4.2 Fuzzy Rules....78
3.4.3 Defuzzification....80
3.4.3.1 Stage 1: Scaling the Membership Functions....81
3.4.3.2 Stage 2: Finding the Centroid....82
3.4.3.3 Defuzzifying at the Extremes....83

3.4.3.4 Sugeno Defuzzification 84
3.4.3.5 A Defuzzification Anomaly 85
3.5 Fuzzy Control Systems 86
3.5.1 Crisp and Fuzzy Control 86
3.5.2 Fuzzy Control Rules 87
3.5.3 Defuzzification in Control Systems 89
3.6 Fuzzy Logic: Type-2 90
3.7 Other Techniques 93
3.7.1 Dempster–Shafer Theory of Evidence 94
3.7.2 Inferno 95
3.8 Summary 95
Further Reading 96

4 Agents, Objects, and Frames 97
4.1 Birds of a Feather: Agents, Objects, and Frames 97
4.2 Intelligent Agents 98
4.3 Agent Architectures 99
4.3.1 Logic-Based Architectures 100
4.3.2 Emergent Behavior Architectures 100
4.3.3 Knowledge-Level Architectures 100
4.3.4 Layered Architectures 101
4.4 Multiagent Systems (MASs) 102
4.4.1 Benefits of a Multiagent System 103
4.4.2 Building a Multiagent System 104
4.4.3 Contract Nets 105
4.4.4 Cooperative Problem-Solving (CPS) 106
4.4.5 Shifting Matrix Management (SMM) 107
4.4.6 Comparison of Cooperative Models 108
4.4.7 Communication between Agents 109
4.5 Swarm Intelligence 111
4.6 Object-Oriented Systems 112
4.6.1 Introducing Object-Oriented Programming (OOP) 112
4.6.2 An Illustrative Example 113
4.6.3 Data Abstraction 114
4.6.3.1 Classes 114
4.6.3.2 Instances 115
4.6.3.3 Attributes (or Data Members) 116
4.6.3.4 Operations (or Methods or Member Functions) 116
4.6.3.5 Creation and Deletion of Instances 117
4.6.4 Inheritance 119
4.6.4.1 Single Inheritance 119
4.6.4.2 Multiple and Repeated Inheritance 120
4.6.4.3 Specialization of Methods 124

4.6.4.4 Class Browsers 124
4.6.5 Encapsulation 125
4.6.6 Unified Modeling Language (UML) 126
4.6.7 Dynamic (or Late) Binding 128
4.6.8 Message Passing and Function Calls 130
4.6.9 Metaclasses 131
4.6.10 Type Checking 132
4.6.11 Persistence 133
4.6.12 Concurrency 133
4.6.13 Active Values and Daemons 134
4.6.14 Summary of Object-Oriented Systems 135
4.7 Objects and Agents 136
4.8 Frame-Based Systems 137
4.9 Summary: Agents, Objects, and Frames 139
Further Reading 139

5 Symbolic Learning 141
5.1 Introduction 141
5.2 Learning by Induction 143
5.2.1 Overview 143
5.2.2 Learning Viewed as a Search Problem 145
5.2.3 Techniques for Generalization and Specialization 146
5.2.3.1 Universalization 147
5.2.3.2 Replacing Constants with Variables 147
5.2.3.3 Using Conjunctions and Disjunctions 148
5.2.3.4 Moving Up or Down a Hierarchy 149
5.2.3.5 Chunking 149
5.3 Case-Based Reasoning (CBR) 150
5.3.1 Storing Cases 151
5.3.1.1 Abstraction Links and Index Links 151
5.3.1.2 Instance-Of Links 152
5.3.1.3 Scene Links 152
5.3.1.4 Exemplar Links 153
5.3.1.5 Failure Links 153
5.3.2 Retrieving Cases 153
5.3.3 Adapting Case Histories 153
5.3.3.1 Null Adaptation 154
5.3.3.2 Parameterization 154
5.3.3.3 Reasoning by Analogy 154
5.3.3.4 Critics 155
5.3.3.5 Reinstantiation 155
5.3.4 Dealing with Mistaken Conclusions 156
5.4 Summary 156
Further Reading 157

6 Single-Candidate Optimization Algorithms..... 159
6.1 Optimization 159
6.2 The Search Space.....160
6.3 Searching the Parameter Space 162
6.4 Hill-Climbing and Gradient-Descent Algorithms 162
6.4.1 Hill-Climbing 162
6.4.2 Steepest Gradient Descent or Ascent 163
6.4.3 Gradient-Proportional Descent or Ascent.....163
6.4.4 Conjugate Gradient Descent or Ascent.....163
6.4.5 Tabu Search.....164
6.5 Simulated Annealing.....164
6.6 Summary 168
Further Reading.....169

7 Genetic Algorithms for Optimization 171
7.1 Introduction: Evolutionary Algorithms 171
7.2 The Basic Genetic Algorithm 172
7.2.1 Chromosomes 172
7.2.2 Algorithm Outline 173
7.2.3 Crossover 173
7.2.4 Mutation.....175
7.2.5 Validity Check.....175
7.3 Selection 176
7.3.1 Selection Pitfalls 176
7.3.2 Fitness-Proportionate Selection 177
7.3.3 Fitness Scaling for Improved Selection.....180
7.3.3.1 Linear Fitness Scaling 180
7.3.3.2 Sigma Scaling 181
7.3.3.3 Boltzmann Fitness Scaling 182
7.3.3.4 Linear Rank Scaling.....183
7.3.3.5 Nonlinear Rank Scaling.....184
7.3.3.6 Probabilistic Nonlinear Rank Scaling.....184
7.3.3.7 Truncation Selection.....185
7.3.3.8 Transform Ranking.....185
7.3.4 Tournament Selection 186
7.3.5 Comparison of Selection Methods.....186
7.4 Elitism 187
7.5 Multiobjective Optimization.....187
7.6 Gray Code 188
7.7 Variable-Length Chromosomes.....189
7.8 Building Block Hypothesis.....191
7.8.1 Schema Theorem.....191
7.8.2 Inversion 191

7.9 Selecting GA Parameters 192
7.10 Monitoring Evolution 192
7.11 Finding Multiple Optima 193
7.12 Genetic Programming (GP) 193
7.13 Other Forms of Population-Based Optimization 194
7.14 Summary 194
Further Reading 195

8 Shallow Neural Networks 197
8.1 Introduction 197
8.2 Neural Network Applications 198
8.2.1 Classification 199
8.2.2 Nonlinear Estimation and Prediction 199
8.2.3 Clustering 200
8.2.4 Memory and Recall 200
8.3 Nodes and Interconnections 201
8.4 Single and Multilayer Perceptrons (SLPs and MLPs) 203
8.4.1 Network Topology 203
8.4.2 Perceptrons as Classifiers 204
8.4.3 Training a Perceptron 208
8.4.4 Hierarchical Perceptrons 211
8.4.5 Buffered Perceptrons 212
8.4.6 Some Practical Considerations 212
8.4.6.1 Overtraining 212
8.4.6.2 Leave-One-Out and K-Fold Cross-Validation 213
8.4.6.3 Data Scaling 214
8.5 Recurrent Networks 215
8.5.1 Simple Recurrent Network (SRN) 215
8.5.2 Hopfield Network 216
8.5.3 Maxnet 218
8.5.4 The Hamming Network 219
8.6 Unsupervised Networks 220
8.6.1 Adaptive Resonance Theory (ART) Networks 220
8.6.2 Kohonen Self-Organizing Networks 222
8.6.3 Radial Basis Function (RBF) Networks 223
8.7 Spiking Neural Networks (SNNs) 226
8.8 Summary 227
Further Reading 228

9 Deep Neural Networks 229
9.1 Deep Learning 229
9.2 Convolutional Neural Networks (CNNs) for Image Recognition 230

9.2.1 Origins....230
9.2.2 Motivation for Convolutional Networks....230
9.2.3 CNN Structure....233
9.2.3.1 Input Layer....233
9.2.3.2 Feature Maps....235
9.2.3.3 Pooling and Classification Layers....237
9.2.4 Pretrained Networks and Transfer Learning....238
9.2.5 CNNs in Context....239
9.3 Generative Networks....239
9.3.1 Generative versus Discriminative Algorithms....239
9.3.2 Autoencoder Networks....240
9.3.3 Generative Adversarial Networks (GANs)....241
9.4 Long Short-Term Memory (LSTM) Networks....242
9.5 Summary....245
Further Reading....245

10 Hybrid Systems....247
10.1 Convergence of Techniques....247
10.2 Blackboard Systems for Multifaceted Problems....248
10.3 Parameter Setting....250
10.3.1 Genetic–Neural Systems....250
10.3.2 Genetic–Fuzzy Systems....251
10.3.3 Fuzzy–Genetic Systems....252
10.4 Capability Enhancement....253
10.4.1 Neuro–Fuzzy Systems....253
10.4.2 Memetic Algorithms: Genetic Algorithms with Local Search....255
10.4.3 Learning Classifier Systems (LCSs)....257
10.5 Clarification and Verification of Neural Network Outputs....257
10.6 Summary....258
Further Reading....258

11 AI Programming Languages and Tools....259
11.1 A Range of Intelligent Systems Tools....259
11.2 Features of AI Languages....261
11.2.1 Lists....261
11.2.2 Other Data Types....262
11.2.3 Programming Environments....264
11.3 Lisp....264
11.3.1 Background....264
11.3.2 Lisp Functions....265
11.3.3 A Worked Example....268
11.4 Prolog....276

11.4.1 Background....276
11.4.2 A Worked Example....277
11.4.3 Backtracking in Prolog....283
11.5 Python....287
11.5.1 Background....287
11.5.2 A Worked Example....288
11.6 Comparison of AI Languages....295
11.7 Summary....295
Further Reading....296

12 Systems for Interpretation and Diagnosis....297
12.1 Introduction....297
12.2 Deduction and Abduction for Diagnosis....299
12.2.1 Exhaustive Testing....300
12.2.2 Explicit Modeling of Uncertainty....301
12.2.3 Hypothesize-and-Test....301
12.3 Depth of Knowledge....302
12.3.1 Shallow Knowledge....302
12.3.2 Deep Knowledge....303
12.3.3 Combining Shallow and Deep Knowledge....305
12.4 Model-Based Reasoning....305
12.4.1 The Limitations of Rules....305
12.4.2 Modeling Function, Structure, and State....306
12.4.2.1 Function....307
12.4.2.2 Structure....308
12.4.2.3 State....310
12.4.3 Using the Model....312
12.4.4 Monitoring....314
12.4.5 Tentative Diagnosis....315
12.4.5.1 The Shotgun Approach....315
12.4.5.2 Structural Isolation....316
12.4.5.3 The Heuristic Approach....316
12.4.6 Fault Simulation....316
12.4.7 Fault Repair....317
12.4.8 Using Problem Trees....317
12.4.9 Summary of Model-Based Reasoning....318
12.5 Case Study: A Blackboard System for Interpreting Ultrasonic Images....319
12.5.1 Ultrasonic Imaging....319
12.5.2 Agents in DARBS....321
12.5.3 Rules in DARBS....323
12.5.4 The Stages of Image Interpretation....325
12.5.4.1 Arc Detection Using the Hough Transform....325

12.5.4.2 Gathering the Evidence 326
12.5.4.3 Defect Classification 328
12.5.5 The Use of Neural Networks 328
12.5.5.1 Defect Classification Using a Neural Network 328
12.5.5.2 Echodynamic Classification Using a Neural Network 330
12.5.5.3 Combining the Two Applications of Neural Networks 330
12.5.6 Rules for Verifying Neural Networks 330
12.6 Summary 331
Further Reading 332

13 Systems for Design and Selection 333
13.1 The Design Process 333
13.2 Design as a Search Problem 336
13.3 Computer-Aided Design 338
13.4 The Product Design Specification (PDS): A Telecommunications Case Study 339
13.4.1 Background 339
13.4.2 Alternative Views of a Network 339
13.4.3 Implementation 340
13.4.4 The Classes 340
13.4.4.1 Network 342
13.4.4.2 Link 342
13.4.4.3 Information Stream 342
13.4.4.4 Site 342
13.4.4.5 Equipment 343
13.4.5 Summary of PDS Case Study 343
13.5 Conceptual Design 343
13.6 Constraint Propagation and Truth Maintenance 346
13.7 Case Study: The Design of a Lightweight Beam 350
13.7.1 Conceptual Design 350
13.7.2 Optimization and Evaluation 352
13.7.3 Detailed Design 356
13.8 Design as a Selection Exercise 356
13.8.1 Overview 356
13.8.2 Merit Indices 357
13.8.3 The Polymer Selection Example 358
13.8.4 Two-Stage Selection 358
13.8.5 Constraint Relaxation 360
13.8.6 A Naive Approach to Scoring 364
13.8.7 A Better Approach to Scoring 366

13.8.8 Case Study: The Design of a Kettle....367
13.8.9 Reducing the Search Space by Classification....368
13.9 Failure Mode and Effects Analysis (FMEA)....370
13.10 Summary....372
Further Reading....373

14 Systems for Planning....375
14.1 Introduction....375
14.2 Classical Planning Systems....376
14.3 Stanford Research Institute Problem Solver (STRIPS)....378
14.3.1 General Description....378
14.3.2 An Example Problem....379
14.3.3 A Simple Planning System in Prolog....382
14.4 Considering the Side Effects of Actions....387
14.4.1 Maintaining a World Model....387
14.4.2 Deductive Rules....387
14.5 Hierarchical Planning....389
14.5.1 Description....389
14.5.2 Benefits of Hierarchical Planning....390
14.5.3 Hierarchical Planning with ABSTRIPS....392
14.6 Postponement of Commitment....397
14.6.1 Partial Ordering of Plans....397
14.6.2 The Use of Planning Variables....399
14.7 Job-Shop Scheduling....400
14.7.1 The Problem....400
14.7.2 Some Approaches to Scheduling....401
14.8 Constraint-Based Analysis (CBA)....402
14.8.1 Constraints and Preferences....402
14.8.2 Formalizing the Constraints....403
14.8.3 Identifying the Critical Sets of Operations....405
14.8.4 Sequencing in the Disjunctive Case....406
14.8.5 Sequencing in the Nondisjunctive Case....406
14.8.6 Updating Earliest Start Times and Latest Finish Times....409
14.8.7 Applying Preferences....411
14.8.8 Using Constraints and Preferences....413
14.9 Replanning and Reactive Planning....414
14.10 Summary....415
Further Reading....417

15 Systems for Control....419
15.1 Introduction....419
15.2 Low-Level Control....420
15.2.1 Open-Loop Control....420

15.2.2 Feedforward Control 421
15.2.3 Feedback Control 422
15.2.4 First- and Second-Order Models 422
15.2.5 Algorithmic Control: The PID Controller 423
15.2.6 Bang-Bang Control 425
15.3 Requirements of High-Level (Supervisory) Control 425
15.4 Blackboard Maintenance 427
15.5 Time-Constrained Reasoning 428
15.5.1 Prioritization of Processes 429
15.5.2 Approximation 429
15.5.2.1 Approximate Search 430
15.5.2.2 Data Approximations 431
15.5.2.3 Knowledge Approximations 431
15.5.3 Single and Multiple Instantiation 432
15.6 Fuzzy Control 433
15.7 The BOXES Controller 435
15.7.1 The Conventional BOXES Algorithm 435
15.7.2 Fuzzy BOXES 441
15.8 Neural Network Controllers 442
15.8.1 Direct Association of State Variables with Action Variables 443
15.8.2 Estimation of Critical State Variables 444
15.9 Statistical Process Control (SPC) 447
15.9.1 Applications 447
15.9.2 Collecting the Data 447
15.9.3 Using the Data 449
15.10 Summary 449
Further Reading 451

16 The Future of Intelligent Systems 453
16.1 Benefits 453
16.2 Trends in Implementation 454
16.3 Intelligent Systems and the Internet 455
16.4 Computational Power 456
16.5 Ubiquitous Intelligent Systems 456
16.6 Ethics 458
16.7 Conclusions 459

References 461

Index 479


Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Preface

About This Book

The term *intelligent systems* is broad, covering a range of artificial intelligence (AI) techniques and the ways in which they can be put into practice. It includes symbolic approaches—in which knowledge is explicitly expressed in words and symbols—and numerical approaches such as neural networks and genetic algorithms. Fuzzy logic can be considered to have a foot in both camps. In fact, many practical intelligent systems are a hybrid of different approaches. Whether any of these systems is really capable of displaying human-like intelligent behavior is a moot point. Nevertheless, they are extremely useful, and they have enabled elegant solutions to a wide variety of difficult problems.

There are plenty of other books available on artificial intelligence and related technologies, but I hope this one is substantially different. It takes a practical view, showing the issues encountered in the development of applied systems. I have deliberately described a wide range of intelligent systems techniques, with the help of realistic problems in engineering and science. The examples included here have been specifically selected for the details of the techniques that they illustrate, rather than merely to survey current practice.

The book can be roughly divided into two parts. Chapters 1 to 11 describe the techniques of artificial intelligence, while Chapters 12 to 15 look at four broad categories of applications. These latter chapters explore in depth the design and implementation issues of applied intelligent systems, together with their advantages and difficulties. The four application areas have much in common, as they all concern automated decision-making, while making the best use of the available information. Chapter 16 rounds off the book by looking at some of the trends and likely future directions for AI.

The Fourth Edition

This new edition is intended to retain the strengths of the earlier editions and to build upon them. The book has been updated throughout, with many new references, while retaining seminal sources wherever their influence remains intact. The biggest single change has been the inclusion of a new chapter on deep neural networks, reflecting the growth of machine learning as the dominant technique for AI in the 2020s. There is also a new section on the use of the Python language, which has become the de facto standard language.

As in the third edition, all the rule-based examples, including those that use uncertainty, are presented in the format of a standard artificial intelligence toolkit. The chosen toolkit comprises Flex™ and Flint™ from Logic Programming Associates Ltd., but the rules can be readily translated for other systems. All the examples have been tested and are available for download from my website.

I hope the book will appeal to a wide readership. In particular, I hope that students will be drawn toward this fascinating area from all scientific and engineering subjects, not just from the computer sciences. Beyond academia, the book will appeal to engineers and scientists who are either building intelligent systems or simply keen to know more about them.

Earlier Editions

The first edition of this book was narrower in focus and was entitled *Knowledge-Based Systems for Engineers and Scientists*. The scope of the second edition was widened to include new chapters on intelligent agents, neural networks, optimization algorithms (especially genetic algorithms), and hybrid systems. The revised title of *Intelligent Systems for Engineers and Scientists* reflected the inclusion of computational intelligence techniques alongside knowledge-based systems.

The first two editions were adopted by the Open University for its course *T396: Artificial Intelligence for Technology*, which ran successfully for 12 years. It gives me great pleasure to think of the large number of students who have been introduced to the world of intelligent systems through the book and the associated Open University course. Partly through having such a keen group of critical readers, I received a lot of useful feedback that helped remove minor errors and clarify the explanations.

The third edition was an evolution to reflect intelligent systems practice. Some of the areas that had grown in importance and were expanded included intelligent agents, fuzzy logic, genetic algorithms, and neural networks. Indeed, genetic algorithms were given their own chapter, separated from single-candidate optimization techniques. For the first time, the book was supported by a website with downloadable code.

Acknowledgements

Many people have helped me, either with this edition or the earlier ones, and I am grateful to them all. The list of people who have helped, directly or indirectly, grows with each edition and includes (in alphabetical order): Mike Brayshaw, David Carpenter, Michele Dimont, Carole Gustafson, Jon Hall, Nicholas Hallam, Jo Herbert, Tony Hirst, David Hopgood, Sue Hopgood, Adam Kowalzyk, Dawn Mesa, Priyanka Mundada, Lars Nolle, Sean Ogden, Obinwa Ozonze, Sara Parkin, Phil Picton, Chris Price, Peter Richardson, Philip Sargent, Navin Sullivan, Joy Watts, Patrick Wong, Neil Woodcock, John Wyzalek, and John Zucker.

As with the previous editions, I remain indebted to my wife for letting me get on with it.

Adrian A. Hopgood
Updates and downloads are available from adrianhopgood.com


Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Author

Dr. Adrian Hopgood is Professor of Intelligent Systems and Director of Future & Emerging Technologies at the University of Portsmouth in the UK. He earned his BSc from the University of Bristol, PhD from the University of Oxford, and MBA from the Open University. After completing his PhD, he joined the AI team of Systems Designers PLC. That experience set the direction of his career toward the investigation of intelligent systems and their practical applications. After leaving Systems Designers, he spent 14 years at the Open University and remains attached as a visiting professor. During that period, he also spent two years at Telstra Research Laboratories in Australia, investigating the role of intelligent systems in telecommunications. He has subsequently worked for Nottingham Trent University, De Montfort University, Sheffield Hallam University, and the University of Liège, before joining the University of Portsmouth. Despite assuming several senior management positions during his career, he has never lost his passion for intelligent systems. His website is adrianhopgood.com.


Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Chapter 1

Introduction

1.1 Artificial Intelligence and Intelligent Systems

Over many centuries, tools of increasing sophistication have been developed to serve the human race. Physical tools such as chisels, hammers, spears, arrows, guns, carts, cars, and aircraft all have their place in the history of civilization. Humanity has also developed tools of communication: spoken language, written language, and the language of mathematics. These tools have not only enabled the exchange and storage of information, but they have also allowed the expression of concepts that simply could not exist outside of language.

The last century saw the arrival of a new tool—the digital computer. Computers are able to perform the same sort of numerical and symbolic manipulations that an ordinary person can, but faster and more reliably. They have therefore been able to remove the tedium from many tasks that were previously performed manually and have allowed the achievement of new feats. Such feats range from huge scientific models to the more familiar online banking facilities.

Although these developments are impressive, the computer is actually only performing quite simple operations, albeit rapidly. In such applications, the computer is still only a complex calculating machine. The intriguing idea now is whether we can build a computer (or a computer program) that can *think* (Turing 1950). As Penrose (1999) has pointed out, most of us are quite happy with machines that enable us to do physical things more easily or more quickly, such as digging a hole or traveling along a freeway. We are also happy to use machines that enable us to do physical things that would otherwise be impossible, such as flying. However, the idea of a machine that can think for us is a huge leap forward in our ambitions, and one that raises many ethical and philosophical questions.

Research in artificial intelligence (or simply AI) is directed toward building such a machine and improving our understanding of intelligence. Here is a

DOI: 10.1201/9781003226277-1

simple definition, adapted from Hopgood (2005) by the insertion of the phrase in parenthesis:

> *Artificial intelligence is the science of mimicking (or exceeding) human mental faculties in a computer.*

The ultimate achievement in this field would be to construct a machine that can mimic or exceed all human mental capabilities, including reasoning, understanding, imagination, recognition, creativity, and emotions. We are a long way from achieving that ambition, but some successes have been achieved in mimicking specific areas of human mental activity. For instance, machines are now able to play chess at the highest level, to interpret spoken sentences, to translate foreign languages, and to diagnose medical complaints. An objection to these claimed successes might be that the machine does not tackle these problems in the same way that a human would. A further objection might be that the field of AI should not be limited to human intelligence, but it should include other animal species too. These objections will not be addressed in this book, which is intended as a guide to practical systems and not a philosophical thesis.

In some specialized fields, like interpreting astronomical images, AI can already exceed human capabilities (Hausen and Robertson 2020). In some other fields, the aim is not to produce the best possible form of AI, but rather to mimic human behavior or capabilities. Some examples include an AI opponent in video games, or a model of human responses in cybersecurity development.

In achieving its successes, research into AI, together with other branches of computer science, has resulted in the development of several useful computing tools that form the basis of this book. These tools have a range of potential applications, but this book emphasizes their use in engineering and science. The tools of particular interest can be roughly divided between *knowledge-based systems* (*KBSs*), *computational intelligence* (*CI*), and hybrid systems. KBSs include expert and rule-based systems, frame-based systems, intelligent agents, and case-based reasoning. CI includes neural networks, genetic algorithms, and other optimization algorithms. Techniques for handling uncertainty, such as fuzzy logic, fit into both categories.

Machine learning describes intelligent systems that improve their performance through experience. Typically, this phrase refers to CI and specifically to complex neural networks that learn to recognize patterns from enormous volumes of data (so-called *big data*), as described in Chapter 9: Deep Neural Networks. However, there are many other forms of data-driven machine learning, described in Chapters 6–8. Furthermore, some KBSs also have the ability to grow their knowledge, and these techniques are covered in Chapter 5: Symbolic Learning.

Intelligent systems is a broad term, covering a range of AI techniques and the ways in which they can be put into practice. They include applied KBSs, CI, machine learning, and their hybrids. Intelligent systems have not solved the problem of building an artificial mind. Indeed, some would argue that they show little, if any,

real intelligence. Nevertheless, they have enabled a range of problems to be tackled that were previously considered too difficult, and they have enabled a large number of other problems to be tackled more effectively. From a pragmatic point of view, these successes alone make intelligent systems interesting and useful.

1.2 A Spectrum of Intelligent Behavior

The definition of AI presented earlier leaves the notion of intelligence rather vague. To explore this further, a spectrum of intelligent behaviors can be drawn based on the level of understanding involved (Hopgood 2003, 2005), as shown in Figure 1.1. The lowest-level behaviors include instinctive reactions, such as withdrawing a hand from a hot object or dodging a projectile. The mid-level behaviors include vision and perception, language and interaction, and common-sense responses to unexpected events. These are capabilities that come naturally to humans, with little conscious thought. High-level behaviors demand specialist expertise such as in the legal requirements of company takeovers or the interpretation of radiological images. Such a spectrum of intelligent behaviors is useful for charting the progress of AI, although it has been criticized for oversimplifying the many dimensions of intelligence (Holmes 2003).

Conventional computing techniques have been developed to handle the low-level decision-making and control needed at the low end of the spectrum. Extremely effective computer systems have been developed for monitoring and controlling a variety of equipment. An example of the close regulation and coordination that is possible is demonstrated by various humanoid robots that show human-like mobility (Sakagami et al. 2002; Firth 2007; Gouaillier et al. 2009; Niemüller et al. 2010). As their capabilities for autonomous thought and understanding are improved through technological development, their behaviors can be expected to expand upward from the lower end of the spectrum.

Early AI research, in contrast, began with problems at the high-level end of the spectrum. Two early applications, for example, concerned the specialist areas of mass spectrometry (Buchanan et al. 1969) and bacterial blood infections (Shortliffe 1976). These early triumphs generated great optimism. If a computer could deal with difficult problems that are beyond the capabilities of most ordinary people, it was assumed that more modest human reasoning would be straightforward. Unfortunately, this assumption is false.

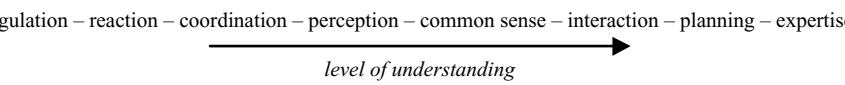


Figure 1.1 A spectrum of intelligent behavior.



Figure 1.2 The challenges of image recognition and interpretation.

The behaviors in the middle of the spectrum, that humans perform with barely a conscious thought, have proven to be the most difficult to emulate in a computer. Consider the photograph in Figure 1.2. Although most of us can recognize the three rabbits in the picture (one of which is a stone statue), the perception involved is an extremely complex behavior. First, recognizing the boundary between objects is difficult. Once an object has been delineated, recognition is far from straightforward. For instance, rabbits come in different shapes, sizes, and colors. They can assume different postures, and they may be partially occluded, like the one in the cage in Figure 1.2. Yet a fully sighted human can perform this perception in an instant, without considering it a particular mark of intelligence. The task's astonishing complexity is revealed by attempting to perform it with a computer (Hopgood 2003, 2005).

1.3 Knowledge-Based Systems (KBSs)

The principal difference between a KBS and a conventional program lies in its structure. In a conventional program, domain knowledge is intimately intertwined with software for controlling the application of that knowledge. In a KBS, the two roles are explicitly separated. In the simplest case, there are two modules: the knowledge module is called the *knowledge base*, and the control module is called the *inference engine* (Figure 1.3). In more complex systems, the inference engine itself may be

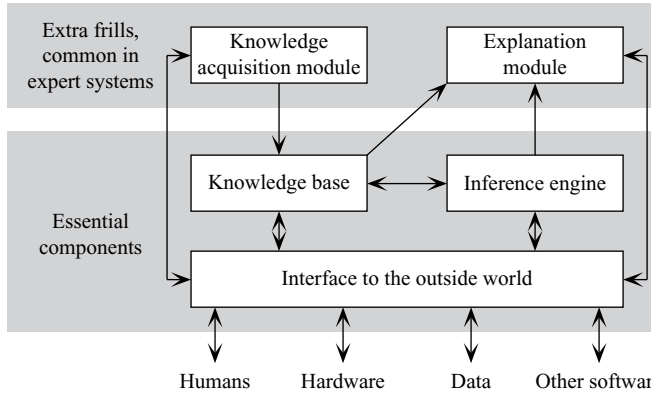


Figure 1.3 The main components of a knowledge-based system.

a KBS containing *metaknowledge*, that is, knowledge of how to apply the domain knowledge.

The explicit separation of knowledge from control of its application makes it easier to add new knowledge, either during program development or in the light of experience during the program's lifetime. There is an analogy with the brain, the control processes of which are approximately unchanging in their nature, like the inference engine. On the other hand, individual behavior is continually modified by new knowledge and experience, like updating the knowledge base.

Suppose that a professional engineer uses a conventional program to support his or her everyday work. Altering the behavior of the program would require him or her to become immersed in the details of the program's implementation. Typically, this would involve altering the control structures of the form:

```
if...then...else...
```

or

```
for x from a to b do...
```

To achieve these changes, the engineer needs to be a proficient programmer. Even if he or she does have this skill, modifications of the kind described are unwieldy and are difficult to make without unwittingly altering some other aspect of the program's behavior.

The KBS approach is more straightforward. The knowledge is represented *explicitly* in the knowledge base, not *implicitly* within the structure of a program. Thus, the knowledge can be altered with relative ease. The inference engine uses the knowledge base to tackle a particular task in a manner that is analogous to a conventional program using a data file.

1.4 The Knowledge Base

1.4.1 Rules and Facts

The knowledge base may be rich with diverse forms of knowledge. Rules and facts are among the most straightforward means of representing knowledge in a knowledge base. Nevertheless, the rules may be complex, and the facts may include sequences, structured entities, attributes of such entities, and the relationships between them. The simplest type of rule is called a production rule and takes the form:

```
if <condition> then <conclusion>
```

The details of the representation used vary from system to system. Using the syntax of the Flex™ KBS toolkit (Melioli et al. 2014; Mutawa and Alzuwawi 2019) and its VisiRule™ visual programming interface (Spenser 2007; Muraina et al. 2016), an example production rule for dealing with the payroll of ACME, Inc., might be:

```
rule r1_1
  if the employer of Person is acme
  then the salary of Person becomes large.
```

Part of the attraction of using production rules is that they can often be written in a form that closely resembles natural language, as opposed to a computer language. The use of capitalization indicates that `Person` is a local variable that can be replaced by a constant value, such as `joe_bloggs` or `mary_smith`, through the process of *pattern matching*. A fact in this KBS might be as follows, where the symbols `/*` and `*/` delineate comments that are ignored by the computer:

```
/* fact f1_1 */
  the employer of joe_bloggs is acme.
```

Facts such as this one may be made available to the KBS at the outset, in which case it is a *static fact*, or while the system is running, in which case it is a *transient fact*. In each case, these are known as *given facts*, as they are given to the KBS. One or more given facts may satisfy the condition of a rule, resulting in the generation of a new fact, known as a *derived fact*. For example, by applying rule r1_1 to fact f1_1, we can derive:

```
/* derived fact f1_2 */
  the salary of joe_bloggs is large.
```

Here, the name `joe_bloggs` from fact f1_1 has been pattern-matched to the variable `Person` from rule r1_1 to produce the new derived fact. The derived fact may satisfy, or partially satisfy, another rule, such as:

```
rule r1_2
  if the salary of Person is large
```

```
or the job_satisfaction of Person is true
then the professional_contentment of Person becomes true.
```

This rule, in turn, may lead to the generation of a new derived fact. Rules r1_1 and r1_2 are interdependent, since the conclusion of one can satisfy the condition of the other.

1.4.2 Inference Networks

The interdependencies among rules, such as r1_1 and r1_2 given in the previous subsection, define a network. Such a network is shown in Figure 1.4 and is known as an *inference network*. The example inference network of Figure 1.4 illustrates the way in which facts can be logically combined to form new facts or conclusions. The facts can be combined logically using `and` and `or`. For example, `and` is used where *happiness* is concerned. For *happiness* to be true, both conditions, namely, *domestic bliss* and *professional contentment,* have to be true. In the inference network, this relationship is shown as a pair of arrows from the two facts to the conclusion *happiness*, with the arrows linked by `and`. *Professional contentment* looks similar, arrows from facts meeting at this conclusion, but they are linked by `or`. Thus *professional contentment* is true, if either *job satisfaction* or *large salary* is true (or both are true).

Another type of connection is `not`. According to Figure 1.4, job satisfaction is achieved through flexibility, responsibility, and the absence of stress. The negative relationship with stress is shown by a dashed line labeled `not`.

An inference network can be constructed by taking facts, and by working out what conditions have to be met for those facts to be true. After these conditions are found, they can be added to the diagram and linked by a logical expression (such

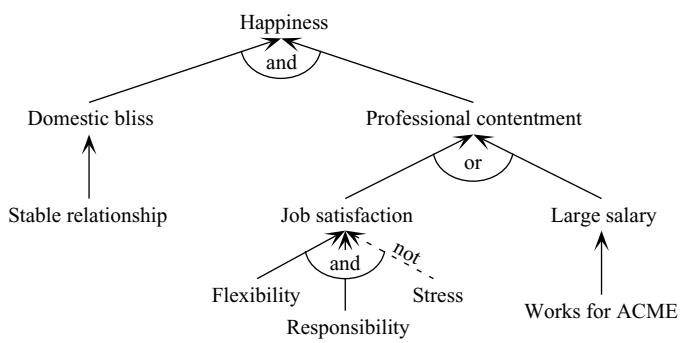


Figure 1.4 An inference network.

as `and`, `or`, `not`). This analysis usually involves breaking down a complex logical expression into smaller parts.

Let us now consider how we might represent these facts and the rules that link them in a conventional program. We might start by creating a data record (a data structure for grouping together different data types) for each employee. Rule r1_1 could be expressed easily enough as a conditional statement (`if...then...`), but it would need to be carefully positioned within the program so that:

- the statement is applied whenever it is needed;
- all relevant variables are in scope (the scope of a variable is that part of the program to which the declaration of the variable applies);
- any values that are assigned to variables remain active for as long as they are needed; and
- the rest of the program is not disrupted.

In effect, the facts and rules are "hard-wired," so that they become an intrinsic part of the program. As many systems need hundreds or thousands of facts and rules, slotting them into a conventional program is a difficult task. This difficulty can be contrasted with a KBS, in which the rule and the fact are represented explicitly and can be changed at will.

Rules such as rule r1_1 are a useful way of expressing many types of knowledge and they are discussed in more detail in Chapter 2. It has been assumed so far that we are dealing with certain knowledge. This is not always the case, and Chapter 3 discusses the use of uncertainty in rules. In the case of rule r1_1, uncertainty may arise from three distinct sources:

- Uncertain evidence.
  (Perhaps we are not certain that Joe Bloggs works for ACME.)
- Uncertain link between evidence and conclusion.
  (We cannot be certain that a randomly selected ACME employee earns a large salary, we just know that it is likely.)
- Vague rule.
  (What is a "large" salary anyway?)

The first two sources of uncertainty can be handled by Bayesian updating of probabilities or variants of this idea. The last source of uncertainty can be handled by fuzzy sets and fuzzy logic. Special types of inference networks can be constructed in which these uncertainties are attached to the inference linkages. *Bayesian inference networks* are an example that will be introduced in Chapter 3.

1.4.3 Semantic Networks

Let us now consider facts in more detail. Facts may be static, in which case they can be written into the knowledge base. Fact f1_1 falls into this category. Note

that static facts need not be permanent, but they change sufficiently infrequently that changes can be accommodated by updating the knowledge base when necessary. In contrast, some facts may be transient. Transient facts (e.g., "Oil pressure is 3000 Nm^{-2}," "the user of this program is Adrian") apply at a specific instance only, or for a single run of the system. The knowledge base may contain defaults that can be used as facts in the absence of transient facts to the contrary. Here is a collection of English-language facts about my car:

- My car is a car (static relationship)
- A car is a vehicle (static relationship)
- A car has four wheels (static attribute)
- A car's speed is 0 mph (default attribute)
- My car is red (static attribute)
- My car is in my garage (default relationship)
- My garage is a garage (static relationship)
- A garage is a building (static relationship)
- My garage is made from brick (static attribute)
- My car is in the High Street (transient relationship)
- The High Street is a street (static relationship)
- A street is a road (static relationship)

Notice that, in this list, we have distinguished between attributes and relationships. Attributes are properties of object *instances* (such as my car) or object *classes* (such as cars and vehicles). Relationships exist among instances of objects and classes of objects. In this way, we can begin to build a model of the subject area of interest, and this reliance on a model will be a recurring topic throughout this book. Attributes and relationships can be represented as a network, known as an *associative network* or *semantic network*, as shown in Figure 1.5.

The semantic network shown in Figure 1.5 illustrates a set of facts. The ovals contain *object instances* (such as `my_car`), *classes of objects* (such as `Vehicle`), or values of *attributes* (such as `red`). Class names such as `Vehicle` are conventionally capitalized in several computer languages, a convention that has been adopted in Figure 1.5. (It is a separate convention from the capitalization of local variable names in the Flex-format rules presented earlier.) The arrows between objects represent relationships between them and are labeled according to their type, such as "is in." Arrows are also used to link objects to their class and to the values of their attributes. So, the attribute *color* labels the link between `my_car` (an object) and `red` (an attribute value). Two ovals joined by a labeled arrow constitute a fact, such as "my car is red."

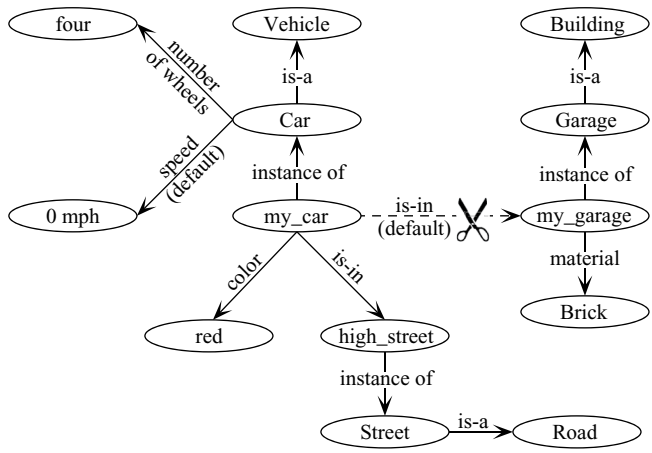


Figure 1.5 A semantic network with an overridden default. Class names are capitalized.

It is important to understand the distinction between object instances and classes of objects:

(a) `Car` and `Vehicle` are both classes of object. They are linked in Figure 1.5 by the relation "is a," which could be expanded to "is a sub-class of." The direction of the arrow is important, indicating that "Car is a Vehicle" and not "Vehicle is a Car."
(b) On the other hand, `my_car` is a unique entity. There may be an identical car somewhere, but they are distinct *instances*. So, the relationship between `my_car` and `Car` takes the form "is an instance of."

At this stage, the semantic network is an unstructured collection of object instances, classes, and attributes. In this representation, attributes are treated in the same way as relationships. In Chapter 4 we will explore agents, objects, and frames, in which relationships and attributes are represented explicitly in a formalized manner. This more structured approach turns these ideas into a powerful programming technique.

1.5 Deduction, Abduction, and Induction

The rules that make up the inference network in Figure 1.4, and the network taken as a whole, are used to link causes and effects:

```
if <cause> then <effect>
```

Using the inference network, we can infer that if Joe Bloggs works for ACME and is in a stable relationship (the causes), then he is happy (the effect). This inference is

the process of *deduction*. Many problems, such as diagnosis, involve reasoning in the reverse direction, that is, we wish to ascertain a cause, given an effect. This process is *abduction*. Given the observation that Joe Bloggs is happy, we can infer by abduction that Joe Bloggs enjoys domestic bliss and professional contentment. However, this conclusion is only valid if the inference network shows *all* of the ways in which a person can find happiness. This limitation is the *closed-world assumption*, the implications of which are discussed in Chapters 2 and 13.

The inference network therefore represents a closed world, where nothing is known beyond its boundaries. As each node represents a possible state of some aspect of the world, a model of the current overall state of the world can be maintained. Such a model is dependent on the extent of the relationships between the nodes in the inference network. In particular, if a change occurs in one aspect of the world, many other nodes could be affected. Determining what else is changed in the world model as a consequence of changing one particular thing is known as the *frame problem*. In the description of Joe Bloggs' world represented in Figure 1.4, this problem is equivalent to determining the extent of the relationships between the nodes. For example, if Joe Bloggs gets a new job, Figure 1.4 suggests that the only direct change is his salary, which could change his professional contentment and happiness. However, in a more complex model of Joe Bloggs' world, many other nodes could also be affected.

If we have many examples of causes and effects, we can infer the rule (or inference network) that links them. For instance, if every employee of ACME that we have met earns a large salary, then we might infer rule r1_1:

```
rule r1_1
  if the employer of Person is acme
  then the salary of Person becomes large.
```

Inferring a rule from a set of example cases of cause and effect is termed *induction*.

We can summarize deduction, abduction, and induction as follows:

- *deduction*: cause + rule ⇒ effect
- *abduction*: effect + rule ⇒ cause
- *induction*: cause + effect ⇒ rule

1.6 The Inference Engine

Inference engines vary greatly according to the type and complexity of knowledge with which they deal. Two important types of inference engines can be distinguished: *forward chaining* and *backward chaining*. These may also be known as *data driven* and *goal driven*, respectively. A KBS working in data-driven mode takes the available information (the "given" facts) and generates as many derived facts as it can. The output is therefore unpredictable. This approach may have either the advantage of

leading to novel or innovative solutions to a problem or the disadvantage of wasting time generating irrelevant information. The data-driven approach might typically be used for problems of interpretation, where we wish to know whatever the system can tell us about some data. A goal-driven strategy is appropriate when a more tightly focused solution is required. For instance, a planning system may be required to generate a plan for manufacturing a consumer product. Any other plans are irrelevant. A backward-chaining system might be presented with this proposition: "a plan exists for manufacturing a widget." It will then attempt to ascertain the truth of this proposition by generating the plan, or it may conclude that the proposition is false and that no plan is possible. Forward and backward chaining are discussed in more detail in Chapter 2. Planning is discussed in Chapter 14.

1.7 Declarative and Procedural Programming

We have already seen that a distinctive characteristic of a KBS is that knowledge is separated from reasoning. Within the knowledge base, the programmer expresses information about the problem to be solved. Often this information is declarative, that is, the programmer states some facts, rules, or relationships without having to be concerned with the detail of *how* and *when* that information is applied. Procedural programming involves telling the computer what to *do*, but declarative programming involves telling the computer what it needs to *know*. The following are three examples of declarative programming:

```
rule r1_3
  if pressure > threshold
  then valve becomes shut.

/* declare a simple fact */
do valve_A becomes shut.

/* declare a fact in the form of a relation */
relation connected (valve_B, tank_3).
```

Each example represents a piece of knowledge that could form part of a knowledge base. The declarative programmer does not necessarily need to state explicitly how, when, and if the knowledge should be used. These details are implicit in the inference engine. An inference engine is normally programmed procedurally; a set of sequential commands is obeyed, which involves extracting and using information from the knowledge base. This task can be made explicit by using *metaknowledge* (knowledge about knowledge), for example,

```
metarule r1_4 /* not Flex format */
  examine rules about valves before rules about pipes.
```

Most conventional programming is procedural. Consider, for example, the following Python program for reading integers from a file and printing them out:

```
# open the file to read integers from a file
# and print them out
filename = input("Please enter file name: ")
file = open(filename, 'r')
try:
  # read the lines of the file
  for line in file:
    # break each line into a list of words
    mylist = line.split()
    for word in mylist:
      try:
        # convert each word to integer format and print
        myinteger = int(word)
        print(myinteger)
      except ValueError:
        # move on if a non-integer is found
        pass
finally:
  # close the file
  file.close()
```

More compact programs could be written, but this version emphasizes that there are explicit step-by-step instructions telling the computer to perform the following actions:

1. Open a data file.
2. Read each line of data from the file in turn.
3. Split each line of data into a list of words.
4. For each word in the list that is a number, convert it to integer format and store it in the variable `myinteger`.
5. Print out the value of `myinteger`.
6. If a word encountered in the list is not a number, move on.
7. Close the data file.

The data file, on the other hand, contains no instructions for the computer at all but just information in the form of a set of integers. The procedural instructions for determining what the computer should do with the integers resides in the program. The data file is therefore declarative, while the program is procedural. The data file is analogous to a trivial knowledge base, and the program is analogous to the corresponding inference engine. Of course, a proper knowledge base would be richer in content, perhaps containing a combination of rules, facts, relations, and data. The corresponding inference engine would be expected to interpret the knowledge, to combine it to form an overall view, to apply the knowledge to data, and to make decisions.

It would be an oversimplification to think that all knowledge bases are written declaratively and that all inference engines are written procedurally. In real systems, a collection of declarative information, such as a rule set, often needs to be embellished by some procedural information. Similarly, there are some inference engines that have been programmed declaratively, notably those implemented in Prolog (described in Chapter 11). Nevertheless, the declarative instructions must eventually be translated into procedural ones, as the computer can only understand procedural instructions at the machine-code level.

1.8 Expert Systems

Expert systems are a type of KBS designed to embody expertise in a particular specialized domain. Example domains might be configuring computer networks, diagnosing faults in machinery, or mineral prospecting. An expert system is intended to act as a human expert who can be consulted on a range of problems that fall within his or her domain of expertise. Typically, the user of an expert system will enter into a dialogue in which he or she describes the problem, such as the symptoms of a fault, and the expert system offers advice, suggestions, or recommendations. The dialogue may be led by the expert system, so that the user responds to a series of questions or enters information into a structured data-entry form. Alternatively, the expert system may allow the user to take the initiative in the consultation by allowing him or her to supply information without necessarily being asked for it.

Since an expert system is a KBS that acts as a specialist consultant, it is often proposed that an expert system must offer certain capabilities that mirror those of a human consultant. In particular, it is often claimed that an expert system must be capable of justifying its current line of inquiry and explaining its reasoning in arriving at a conclusion. This is the purpose of the explanation module in Figure 1.3. As the knowledge in an expert system is explicitly represented, the system can, at the very least, produce a trace of the facts and rules that have been used. The ability of a system to explain its decisions is closely linked to trust in that system. It is an advantage that KBSs have over computational intelligence, where the reasoning is more opaque.

An *expert system shell* is an expert system with an empty knowledge base. These shells are sold as software packages of varying complexity. In principle, it should be possible to buy an expert system shell, build up a knowledge base, and thereby produce an expert system. However, all domains are different, and it is difficult for a software supplier to build a shell that adequately handles them all. The best shells are flexible in their ability to represent and apply knowledge. Without this flexibility, it may be necessary to generate rules and facts in a convoluted style in order to fit the syntax or to force a certain kind of behavior from the system. This situation is not much better than building a conventional program.

1.9 Knowledge Acquisition

The representation of knowledge in a knowledge base can only be addressed once the knowledge is known. There are three distinct approaches to acquiring the relevant knowledge for a particular domain:

- The knowledge is teased out of a domain expert.
- The builder of the KBS *is* a domain expert.
- The system learns automatically from examples.

The first approach is commonly used, but it is fraught with difficulties. The person who extracts the knowledge from the expert and encodes it in the knowledge base is termed the *knowledge engineer*. Typically, the knowledge engineer interviews one or more domain experts and tries to make them articulate their in-depth knowledge in a manner that the knowledge engineer can understand. The inevitable communication difficulties can be avoided by the second approach, in which the domain expert becomes a knowledge engineer, or the knowledge engineer becomes a domain expert.

Finally, there are many circumstances in which the knowledge is either unknown or cannot be expressed explicitly. In these circumstances it may be preferable to have the system generate its own knowledge base from a set of examples. Techniques for automatic learning are discussed in Chapters 5–10.

1.10 Search

Search is the key to practically all problem-solving tasks and has been a major focus of research in intelligent systems. Problem-solving concerns the search for a solution. The detailed engineering applications discussed in this book include the search for a design, plan, control action, or diagnosis of a fault. All of these applications involve searching through the possible solutions, which comprise the *search space*, to find one or more that are optimal or satisfactory. Search is also a key issue for the internal workings of a KBS. The knowledge base may contain hundreds or thousands of rules and facts. The principal role of the inference engine is to search for the most appropriate item of knowledge to apply at any given moment (see Chapter 2).

In the case of searching the knowledge base, it is feasible (although not very efficient) to test all of the alternatives before selecting one. This approach is known as *exhaustive search*. In the search space of an application such as design or diagnosis, exhaustive search is likely to be impractical since the number of candidate solutions is so vast. A practical search therefore has to be selective. To this end, the candidate solutions that make up the search space can be organized as a *search tree*.

The expression *search tree* indicates that solutions can be categorized in some fashion, so that similar solutions are clustered together on the same branch of the tree. Figure 1.6

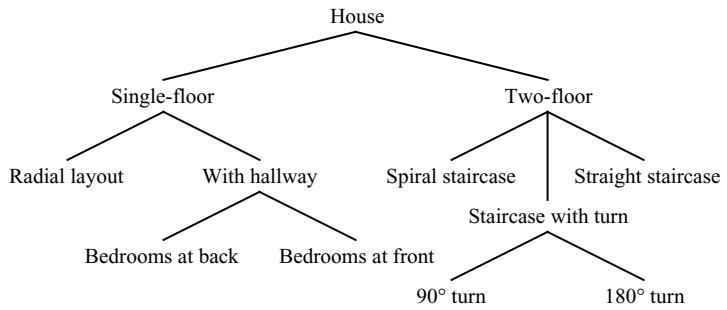


Figure 1.6 A search tree for house designs.

shows a possible search tree for designing a house. Unlike a real tree, the tree shown here has its root at the top and its leaves at the bottom. As we progress toward the leaves, the differences between the designs become less significant. Each alternative design is either generated automatically or found in a database, and then tested for suitability. This approach is described as a *generate-and-test* strategy. If a solution passes the test, the search may continue in order to find further acceptable solutions, or it may stop.

Two alternative strategies for systematically searching the tree are depth-first and breadth-first searches. An example of *depth-first* search is shown in Figure 1.7. In this example, a progressively more detailed description of a single-floor house is built up and tested before other classifications, such as a two-floor house, are considered. When a node fails the test, the search resumes at the previous node where a branch was selected. This process of *backtracking* (see Chapters 2 and 11) is indicated in Figure 1.7 by the broken arrows, which are directed toward the root rather than the leaves of the tree.

In contrast, the *breadth-first* approach (Figure 1.8) involves examining all nodes at a given level in the tree before progressing to the next level. Each node is a

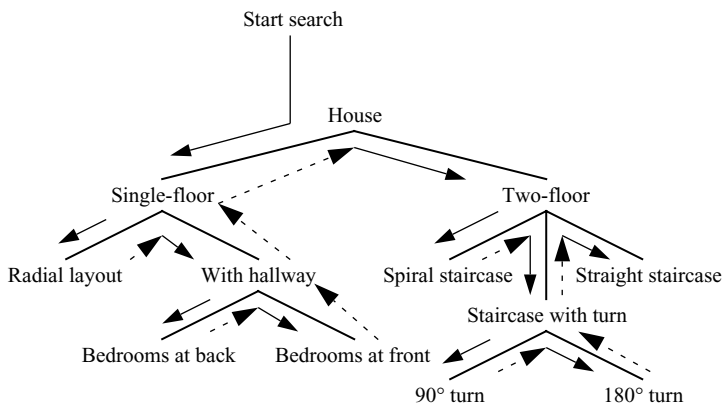


Figure 1.7 Depth-first search (the broken arrows indicate backtracking).

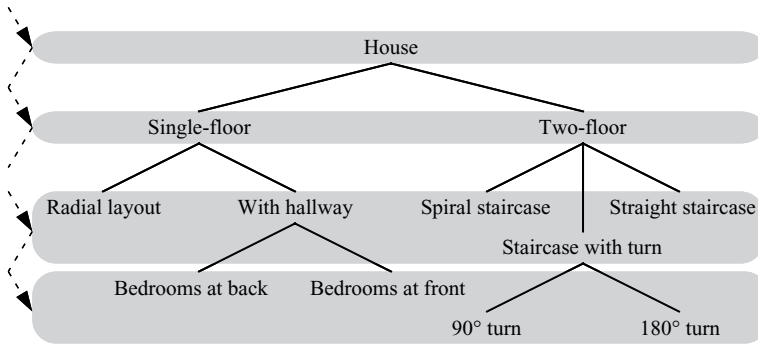


Figure 1.8 Breadth-first search.

generalization of the nodes on its subbranches. Therefore, if a node fails the test, all subcategories are assumed to fail the test and are eliminated from the search.

The systematic search strategies described in the previous paragraphs are examples of *blind search*. The search can be made more efficient either by eliminating unfeasible categories ("pruning the search tree") or by ensuring that the most likely alternatives are tested before less likely ones. To achieve either of these aims, we need to apply heuristics to the search process. Blind search is thereby modified to *heuristic search*. Barr and Feigenbaum (1986) have surveyed the use of the word *heuristic* and produced this general description:

> *A heuristic is a rule of thumb, strategy, trick, simplification, or any other kind of device that drastically limits search for solutions in large search spaces. Heuristics do not guarantee optimal solutions; in fact they do not guarantee any solution at all; all that can be said for a useful heuristic is that it offers solutions that are good enough most of the time.*

In a diagnostic system for plumbing, a useful heuristic may be that pipes are more likely to leak at joints than along lengths. This heuristic defines a search strategy, namely, to look for leaking joints first. In a design system for a house, we might use a heuristic to eliminate all designs that would require us to walk through a bedroom to reach the bathroom. In a short-term planning system, a heuristic might be used to ensure that no plans look ahead more than a week. In a control system for a nuclear reactor, a heuristic may ensure that the control rods are never raised while the coolant supply is turned off.

1.11 Computational Intelligence (CI)

The discussion so far has mostly concentrated on types of KBSs. They are symbolic representations, in which knowledge is explicitly represented in words and symbols that are combined to form rules, facts, relations, or other forms of knowledge

representation. As the knowledge is explicitly written, it can be read and understood by a human. As remarked in Section 1.8, this explicit representation of knowledge contributes to the system's *explainability* and the trust that can be placed in it.

These symbolic techniques (Chapters 2, 4, and 5) contrast with numerical techniques such as single-candidate optimization algorithms (Chapter 6), genetic algorithms (Chapter 7), shallow neural networks (Chapter 8), and deep neural networks (Chapter 9). There, the knowledge is not explicitly stated but is represented by numbers that are adjusted as the system improves its accuracy. Those techniques are collectively known as *CI* or *soft computing*. CI comprises data-driven forms of AI, whereas KBSs are knowledge-driven. As remarked in Section 1.1, *machine learning* is a broad term for any intelligent systems that improve their performance through experience. Nevertheless, the term is now most commonly used to describe CI techniques based around so-called *deep learning*, using deep neural networks and large data sets known as big data.

Chapter 3 describes three techniques for handling uncertainty: Bayesian updating, certainty factors, and fuzzy logic. These techniques all use a mixture of rules and associated numerical values, and they can therefore be considered as both CI tools and KBS tools. As Figure 1.9 shows, CI embraces:

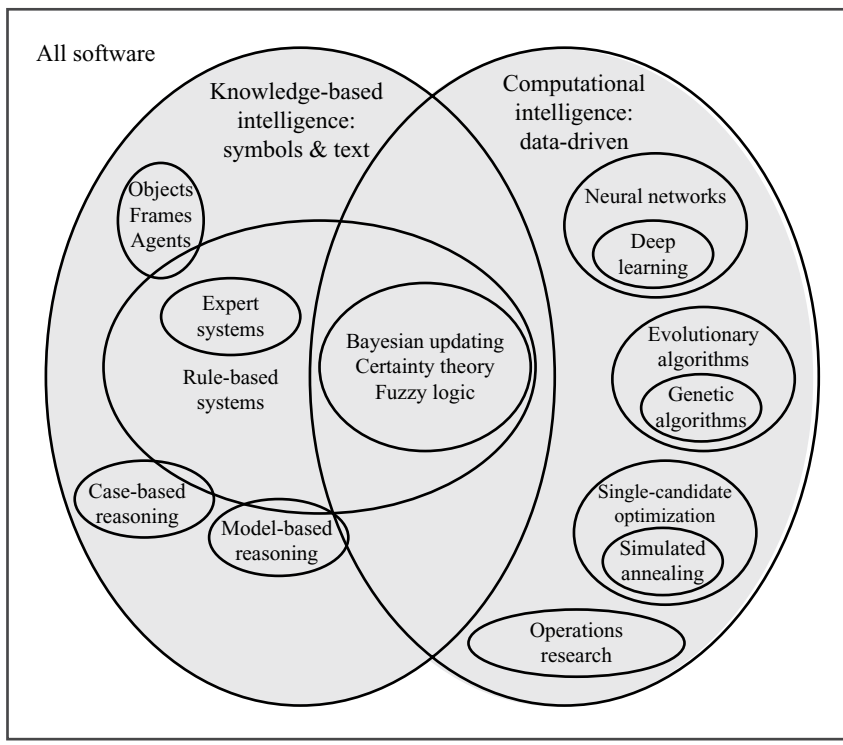


Figure 1.9 Categories of AI software.

- neural networks, including the deep neural networks most commonly associated with machine learning and big data;
- optimization algorithms including evolutionary algorithms, which in turn include genetic algorithms;
- Operations research (OR) that uses data analytics, modeling, and optimization to find solutions to problems. It predates AI as a discipline, but can be regarded as a form of CI;
- probabilistic methods such as Bayesian updating and certainty factors;
- fuzzy logic; and
- combinations of these techniques with each other and with KBSs.

1.12 Integration with Other Software

This book will take a broad view of AI and intelligent systems. It will stress the interrelationship of the various KBS and CI techniques with each other and with conventional programming (Figure 1.9). These techniques do not necessarily represent exclusive alternatives but can often be used cooperatively. For example, a designer may already have an excellent conventional program for simulating the aerodynamic properties of a car. In this case, a KBS might be used as an interface to the program, allowing the program to be used to its full potential. Similarly, an autonomous vehicle might use neural networks for image recognition, but a KBS for applying the rules of the road.

Chapter 10 describes some of the ways in which intelligent systems techniques can be used cooperatively within hybrid systems, and Chapter 11 introduces three AI programming languages. Chapters 12 to 15 describe some practical applications, most of which are hybrids of some form. If a problem can be broken down into subtasks, a blackboard system (described in Chapters 10 and 12) might provide a suitable way of tackling it. Blackboard systems allow each subtask to be handled using an appropriate technique, thereby contributing most effectively to the overall solution.

As with any other technique, KBSs and CI are not suitable for all types of problems. Each problem calls for the most appropriate tool, but KBSs, CI, and hybrids of the two can be used for many problems that would be impracticable by other means.

Further Reading

Akerkar, R. A., and P. S. Sajja. 2009. *Knowledge Based Systems*. Jones and Bartlett, Sudbery, MA.

Burkov, A. 2019. *The Hundred-Page Machine Learning Book*. Andriy Burkov, Quebec City, Canada.

Carter, M. 2007. *Minds and Computers: An Introduction to the Philosophy of Artificial Intelligence*. Edinburgh University Press, Edinburgh, UK.

Engelbrecht, A. P. 2007. *Computational Intelligence: An Introduction*. 2nd ed. John Wiley & Sons, New York, NY.

Luger, G. F. 2008. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*. 6th ed. Pearson, Harlow, UK.

Mitchell, M. 2019. *Artificial Intelligence: A Guide for Thinking Humans*. Pelican Books, London, UK.

Negnevitsky, M. 2011. *Artificial Intelligence: A Guide to Intelligent Systems*. 3rd ed. Addison-Wesley, Reading, MA.

Russell, S., and P. Norvig. 2016b. *Artificial Intelligence: a Modern Approach*. 3rd ed. Pearson Education, London, UK.

Introduction

Akerkar, R. A. , and P. S. Sajja . 2009. Knowledge Based Systems. Jones and Bartlett, Sudbery, MA.

Burkov, A. 2019. The Hundred-Page Machine Learning Book. Andriy Burkov, Quebec City, Canada.

Carter, M. 2007. Minds and Computers: An Introduction to the Philosophy of Artificial Intelligence. Edinburgh University Press, Edinburgh, UK.

Engelbrecht, A. P. 2007. Computational Intelligence: An Introduction. 2nd ed. John Wiley & Sons, New York, NY.

Luger, G. F. 2008. Artificial Intelligence: Structures and Strategies for Complex Problem Solving. 6th ed. Pearson, Harlow, UK.

Mitchell, M. 2019. Artificial Intelligence: A Guide for Thinking Humans. Pelican Books, London, UK.

Negnevitsky, M. 2011. Artificial Intelligence: A Guide to Intelligent Systems. 3rd ed. Addison-Wesley, Reading, MA.

Russell, S. , and P. Norvig . 2016b. Artificial Intelligence: a Modern Approach. 3rd ed. Pearson Education, London, UK.

Rule-Based Systems

Akerkar, R. A. , and P. S. Sajja . 2009. Knowledge Based Systems. Jones and Bartlett, Sudbery, MA.

Brachman, R. , and H. Levesque . 2011. Knowledge Representation and Reasoning. Elsevier, San Francisco, CA.

Darlington, K. W. 2000. The Essence of Expert Systems. Prentice Hall, Upper Saddle River, NJ.

Giarratano, J. C. , and G. D. Riley . 2004. Expert Systems: Principles and Programming. 4th ed. Course Technology Inc., Boston, MA.

Jannach, D. , M. Zanker , A. Felfernig , and G. Friedrich . 2011. Recommender Systems: An Introduction. Cambridge University Press, Cambridge, UK.

Handling Uncertainty: Probability and Fuzzy Logic

Bouchon-Meunier, B. , C. Marsala , M. Rifqi , and R. R. Yager , eds. 2008. Uncertainty and Intelligent Information Systems. World Scientific, Hong Kong.

Chen, G. and T. T. Pham . 2019. Introduction to Fuzzy Sets, Fuzzy Logic, and Fuzzy Control Systems. CRC Press, Boca Raton, FL.

Halpern, J. Y. 2017. Reasoning about Uncertainty. 2nd ed. MIT Press, Cambridge MA.

Jantzen, J. 2013. Foundations of Fuzzy Control. 2nd ed. John Wiley & Sons, Chichester, UK.

Ross, T. J. 2016. Fuzzy Logic with Engineering Applications. 4th ed. John Wiley & Sons, Chichester, UK.

Agents, Objects, and Frames

Bennett, S. , S. McRobb , and R. Farmer . 2010. Object-Oriented Systems Analysis and Design using UML. 4th ed. McGraw-Hill, London, UK.

Booch, G. , R. A. Maksimchuk , M. W. Engel , B. J. Young , J. Conallen , and K. A. Houston . 2007. Object-oriented Analysis and Design with Applications. 3rd ed. Addison-Wesley, Reading, MA.

Shoham, Y. , and K. Leyton-Brown . 2009. Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations. Cambridge University Press, Cambridge, UK.

Stroustrup, B . 2014. Programming: Principles and Practice using C++. 2nd ed. Addison-Wesley, Reading, MA.
Weiss, G. , ed. 2017. Multiagent Systems. 2nd ed. MIT Press, Cambridge, MA.
Wooldridge, M. J. 2009. An Introduction to Multiagent Systems. 2nd ed. Wiley, Chichester, UK.

Symbolic Learning

Alpaydin, E. 2010. Introduction to Machine Learning. 2nd ed. MIT Press, Cambridge, MA.
Kolodner, J. 1993. Case-Based Reasoning. Morgan Kaufmann, San Francisco, CA.
Langley, P. 1995. Elements of Machine Learning. Morgan Kaufmann, San Francisco, CA.
Leeland, A. M. 2010. Case-Based Reasoning: Processes, Suitability & Applications. Nova Science, Hauppauge, NY.
Mitchell, T. M. 1997. Machine Learning. McGraw-Hill, New York.
Richter, M. M. and R. O. Weber . 2013. Case-Based Reasoning: A Textbook. Springer-Verlag, Berlin, Germany.
Watson, I. D. 1997. Applying Case-based Reasoning: Techniques for Enterprise Systems. Morgan Kaufmann, San Francisco, CA.

Single-Candidate Optimization Algorithms

Antoniou, A. , and W.-S. Lu . 2021. Practical Optimization: Algorithms and Engineering Applications. 2nd ed. Springer, Berlin, Germany.
Kochenderfer, M. J. and Wheeler, T. A. 2019. Algorithms for Optimization. MIT Press, Cambridge, MA.
van Laarhoven, P. J. M. 2010. Simulated Annealing: Theory and Applications. Kluwer, Dordrecht, Netherlands.

Genetic Algorithms for Optimization

Davis, L. , ed. 1991. Handbook of Genetic Algorithms. Van Nostrand Reinhold, New York.
Goldberg, D. E. 1989. Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, Reading, MA.
Goldberg, D. E. , and K. Sastry . 2010. Genetic Algorithms: The Design of Innovation. 2nd ed. Springer, Berlin, Germany.
Haupt, R. L. 2004. Practical Genetic Algorithms. 2nd ed. Wiley-Blackwell, London, UK.
Johnson, J. H. , and P. D. Picton . 1995. Concepts in Artificial Intelligence. Butterworth-Heinemann, Oxford, UK.
Mitchell, M. 1996. An Introduction to Genetic Algorithms. MIT Press, Cambridge, MA.
Wirsansky, W. 2020. Hands-On Genetic Algorithms With Python. Packt Publishing, Birmingham, UK.

Shallow Neural Networks

Bishop, C. M. 1995. Neural Networks for Pattern Recognition. Clarendon Press, Oxford, UK.
Gurney, K. 1997. An Introduction to Neural Networks. CRC Press, Boca Raton, FL.
Haykin, S. 2008. Neural Networks and Learning Machines. 3rd ed. Pearson Education, Upper Saddle River, NJ.

Picton, P. D. 2000. Neural Networks. 2nd ed. Palgrave Macmillan, New York, NY.
Taylor, M. 2017. Neural Networks: A Visual Introduction for Beginners, Blue Windmill Media, Sheffield, UK.

Deep Neural Networks

Aggarwal, C. C. 2018. Neural Networks and Deep Learning. Springer, Cham, Switzerland.
Alpaydin, E. 2016. Machine Learning: The New AI. MIT Press, Cambridge, MA.
Chollet, F. 2018. Deep Learning With Python. Manning, Shelter Island, NY.
Goodfellow, I. , Y. Bengio , A. Courville , and F. Bach . 2017. Deep Learning. MIT Press, Cambridge, MA.
Kelleher, J. D. 2019. Deep Learning. MIT Press, Cambridge, MA.
Michelucci, U. 2018. Applied Deep Learning: A Case-Based Approach to Understanding Deep Neural Networks. Apress, Dübendorf, Switzerland.

Hybrid Systems

Castillo, O. , and P. Melin , eds. 2020. Hybrid Intelligent Systems in Control, Pattern Recognition and Medicine. Springer Nature, Cham, Switzerland.
Zgurovsky, M. , V. Sineglazov , and E. Chumachenko . 2020. Artificial Intelligence Systems Based on Hybrid Neural Networks: Theory and Applications. Springer Nature, Cham, Switzerland.

AI Programming Languages and Tools

Barski, C. 2010. Land of Lisp. No Starch Press, San Francisco, CA.
Kreiker, P. 2000. Visual Lisp: Guide to Artful Programming. Delmar, Albany, NY.
Seibel, P. 2011. Practical Common Lisp. Apress, New York.
Touretzky, D. S. 2013. Common Lisp: A Gentle Introduction to Symbolic Computation. Dover Publications, New York.
Watson, M. 2020. Loving Common Lisp, or the Savvy Programmer's Secret Weapon. 6th ed. Leanpub, Victoria, Canada.
Blackburn, P. , Bos, J. , and Striegnitz, K. 2006. Learn Prolog Now! College Publications, London.
Bramer, M. 2013. Logic Programming with Prolog. 2nd ed. Springer, New York.
Bratko, I. 2011. Prolog Programming for Artificial Intelligence. 4th ed. Addison-Wesley, Reading, MA.
Clocksin, W. F. and Mellish, C. S. 2013. Programming in Prolog: Using the ISO Standard. 5th ed. Springer, New York.
Scott, R. 2010. A Guide to Artificial Intelligence with Visual Prolog. Outskirts Press, Denver, CO.
Barry, P. 2016. Head First Python, 2nd ed. O'Reilly, Sebastopol, CA.
Cannon, J. 2014. Python Programming for Beginners. CreateSpace, Scotts Valley, CA.
Downey, A. B. 2015. Think Python: How to Think Like a Computer Scientist, O'Reilly, Sebastopol, CA.
Matthes, E. 2019. Python Crash Course, 2nd ed. No Starch Press, San Francisco, CA.
Shaw, Z. A. 2017. Learn Python 3 the Hard Way. Addison-Wesley, Boston, MA.

Systems for Interpretation and Diagnosis

Ding, S. X. 2013. Model-based Fault Diagnosis Techniques: Design Schemes, Algorithms, and Tools. 2nd ed. Springer, Berlin, Germany.

Korbicz, J. , J. M. Koscielny , Z. Kowalczuk , and W. Cholewa , eds. 2004. Fault Diagnosis: Models, Artificial Intelligence, Applications. Springer, Berlin, Germany.

Pascual, D. G. 2015. Artificial Intelligence Tools: Decision Support Systems in Condition Monitoring and Diagnosis, CRC Press, Boca Raton, FL.

Price, C. J. 2000. Computer-Based Diagnostic Systems. Springer-Verlag, London, UK.

Systems for Design and Selection

Gero, J. S. , ed. 2011. Design Computing and Cognition '10. Springer, Berlin, Germany.

Goel, A. K. , and A. G. de Silva Garza . 2010. Special issue on artificial intelligence in design. Journal of Computing and Information Science in Engineering 10 (3).

Noor, A. K. 2017. AI and the Future of Machine Design, Mechanical Engineering 139(10): 38–43.

Systems for Planning

Geffner, H. , and B. Bonet . 2013. A Concise Introduction to Models and Methods for Automated Planning. Morgan & Claypool, San Rafael, CA.

Ghallab, M. , D. Nau , and Traverso, P. 2004. Automated Planning: Theory and Practice. Morgan Kaufmann, San Mateo, CA.

Ghallab, M. , D. Nau , and Traverso, P . 2016. Automated Planning and Acting. Cambridge University Press, New York, NY.

van Wezel, W. , R. J. Jorna , and A. M. Meystel . 2006. Planning in Intelligent Systems: Aspects, Motivations, and Methods. John Wiley & Sons, Hoboken, NJ.

Vlahavas, I. , and D. Vrakas . 2004. Intelligent Techniques for Planning. IGI Publishing, Hershey, PA.

Systems for Control

Franklin, G. F. , J. D. Powell , and A. Emami-Naeini . 2020b. Feedback Control of Dynamic Systems. 8th ed. Pearson Education, Upper Saddle River, NJ.

Jantzen, J. 2013. Foundations of Fuzzy Control: A Practical Approach, 2nd ed. John Wiley & Sons, Chichester, UK.

Nanayakkara, T. , F. Sahin , and M. Jamshidi . 2009. Intelligent Control Systems with an Introduction to System of Systems Engineering. CRC Press, Boca Raton, FL.

Suykens, J. A. K. , J. P. L. Vandewalle , and B. L. R. De Moor . 2010. Artificial Neural Networks for Modelling and Control of Non-Linear Systems. Kluwer, Dordrecht, Germany.

References

Aamodt, A. , and E. Plaza . 1994. Case-based reasoning – Foundational issues, methodological variations, and system approaches. AI Communications 7 (1):39–59.

AbdelMeguid, H. , and B. Ulanicki . 2010. Feedback rules for operation of pumps in a water supply system considering electricity tariffs. In WDSA 2010:12th annual Water Distribution Systems Analysis conference. Tucson, AZ.

Abe, S. 2010. Support Vector Machines for Pattern Classification. 2nd ed., Advances in Pattern Recognition. Springer, London, UK.

Aineto, D. , S. Jiménez , and E. Onaindia . 2018. Learning STRIPS action models with classical planning. In 28th International Conference on Automated Planning and Scheduling (ICAPS 2018), 399–407.

Al-Fuqaha, A. , M. Guizani , M. Mohammadi , M. Aledhari , and M. Ayyash . 2015. Internet of Things: A survey on enabling technologies, protocols, and applications. IEEE Communications Surveys & Tutorials 17 (4):2347–2376.

Al-Kuzee, J. , T. Matsuura , A. Goodyear , L. Nolle , A. A. Hopgood , P. D. Picton , and N. St. J. Braithwaite . 2003. Intelligent control of low-pressure plasma processing. In IEEE Industrial Electronics Society Conference (ICON03), 2, 1932–1937.

Albashiri, K. A. , F. Coenen , and P. Leng . 2009. EMADS: An extendible multi-agent data miner. Knowledge Based Systems 22 (7):523–528.

Alpaydin, E. 2010. Introduction to Machine Learning. 2nd ed. MIT Press, Cambridge, MA.

AltInten, A. , F. Ketevanlioglu , S. Erdogan , H. Hapoglu , and M. Alpbaz . 2008. Self-tuning PID control of jacketed batch polystyrene reactor using genetic algorithm. Chemical Engineering Journal 138 (1–3):490–497.

Altug, S. , M. Y. Chow , and H. J. Trussell . 1999. Heuristic constraints enforcement for training of and rule extraction from a fuzzy/neural architecture – Part II: Implementation and application. IEEE Transactions on Fuzzy Systems 7 (2):151–159.

Ampratwum, C. S. , P. D. Picton , and A. A. Hopgood . 1997. A rule-based system for optical emission spectral analysis. In Symposium on industrial applications of Prolog (INAP'97). Kobe, Japan. 99–102.

Aoki, H. , and K. Sasaki . 1990. Group supervisory control system assisted by artificial intelligence. Elevator World, February, 70–91.

Arroyo-Figueroa, G. , Y. Alvarez , and L. E. Sucar . 2000. SEDRET – An intelligent system for the diagnosis and prediction of events in power plants. Expert Systems with Applications 18:75–86.

Ashby, M. F. 1989. On the engineering properties of materials. Acta Metallurgica 37:1273–1293.

Baker, J. E. 1985. Adaptive selection methods for genetic algorithms. In International Conference on Genetic Algorithms and their Application. 101–111.

Balakrishnan, A. , and T. Semmelbauer . 1999. Circuit diagnosis support system for electronics assembly operations. Decision Support Systems 25:251–269.

Baldwin, J. M. 1896. A new factor in evolution. American Naturalist 30 (354):441–451.

Bandyopadhyay, S. , S. Saha , U. Maulik , and K. Deb . 2008. A simulated annealing-based multiobjective optimization algorithm: AMOSA. IEEE Transactions on Evolutionary Computation 12 (3):269–283.

Barnett, J. A. 1981. Computational methods for a mathematical theory of evidence. In 7th International Joint Conference on Artificial Intelligence (IJCAI'81). Vancouver, Canada. 868–875.

Barr, A. , and E. A. Feigenbaum . 1986. The Handbook of Artificial Intelligence, vol. 1. Addison-Wesley, Reading, MA.

Bartak, R. , M. A. Salido , and F. Rossi . 2010. Constraint satisfaction techniques in planning and scheduling. Journal of Intelligent Manufacturing 21 (1):5–15.

Beaudoin, J. F. , S. Delisle , M. Dugre , and J. St-Pierre . 2005. Reengineering the knowledge component of a data warehouse-based expert diagnosis system. In Database and Expert Systems Applications, Lecture Notes in Computer Science, vol. 3588, edited by K. V. Andersen , J. Debenham and R. Wagner . Springer-Verlag, Berlin, Germany. 910–919.

Bel, G. , E. Bensana , D. Dubois , J. Erschler , and P. Esquirol . 1989. A knowledge-based approach to industrial job-shop scheduling. In Knowledge-based Systems in Manufacturing, edited by A. Kusiak . Taylor and Francis, Philadelphia, PA. 207–246.

Bellifemine, F. , A. Poggi , and G. Rimassi . 1999. JADE: A FIPA-compliant agent framework. In Proc. Practical Applications of Intelligent Agents and Multi-Agents. 97–108.
Bennett, M. E. 1987. Real-time continuous AI. IEE Proceedings-D 134 (4):272–277.
Bennett, S. , S. McRobb , and R. Farmer . 2010. Object-oriented Systems Analysis and Design using UML. 4th ed. McGraw-Hill, London, UK.
Bernard, J. A. 1988. Use of a rule-based system for process control. IEEE Control Systems Magazine 8 (5):3–13.
Berstel, B. 2002. Extending the RETE Algorithm for event management. In Proceedings of Ninth International Symposium on Temporal Representation and Reasoning. 49–51.
Bichindaritz, I. and C. Marling . 2006. Case-based reasoning in the health sciences: What's next? Artificial Intelligence in Medicine 36(2):127–135.
Booch, G. , J. Rumbaugh , and I. Jacobson . 2005. The Unified Modeling Language User Guide. 2nd ed. Addison-Wesley, Reading, MA.
Booch, G. , R. A. Maksimchuk , M. W. Engel , B. J. Young , J. Conallen , and K. A. Houston . 2007. Object-oriented Analysis and Design with Applications. 3rd ed. Addison-Wesley, Reading, MA.
Bratko, I . 2011. Prolog Programming for Artificial Intelligence. 4th ed. Addison-Wesley, Reading, MA.
Bratman, M. E. 1987. Intentions, Plans, and Practical Reason. Harvard University Press, Cambridge, MA.
Bratman, M. E. , D. J. Israel , and M. E. Pollack . 1988. Plans and resource-bounded practical reasoning. Computational Intelligence 4:349–355.
Brooks, R. A. 1991. Intelligence without reason. In 12th International Joint Conference on Artificial Intelligence (IJCAI'91). Sydney, Australia. 569–595.
Brown, D. , and B. Chandrasekaran . 1985. Expert systems for a class of mechanical design activity. In Knowledge Engineering in Computer-aided Design, edited by J. Gero . Elsevier, Amsterdam, Netherlands. 259–282.
Bruninghaus, S. , and K. D. Ashley . 2003. Combining case-based and model-based reasoning for predicting the outcome of legal cases. Lecture Notes in Artificial Intelligence 2689:65–79.
Bruno, G. , A. Elia , and P. Laface . 1986. A rule-based system to schedule production. IEEE Computer 19 (7):32–40.
Brzykcy, G. , J. Martinek , A. Meissner , and P. Skrzypczynski . 2001. Multi-agent blackboard architecture for a mobile robot. In IEEE/RSJ International Conference on Intelligent Robots and Systems. 2369–2374.
Buchanan, B. G. , and R. O. Duda . 1983. Principles of rule-based expert systems. In Advances in Computers, edited by M. C. Yovits . Academic Press, New York, NY.
Buchanan, B. G. , G. L. Sutherland , and E. A. Feigenbaum . 1969. Heuristic DENDRAL: A program for generating explanatory hypotheses in organic chemistry. Machine Intelligence 4: 209–254.
Bykat, A . 1990. NICBES-2, a nickel-cadmium battery expert system. Applied Artificial Intelligence 4:133–141.
Caldas, L. G. , and L. K. Norford . 2002. A design optimization tool based on a genetic algorithm. Automation in Construction 11 (2):173–184.
Campolucci, P. , S. Fiori , A. Uncini , and F. Piazza . 1997. A new IIR-MLP learning algorithm for on-line signal processing. In IEEE Int. Conf. on Acoustics, Speech, and Signal Processing ICASSP-97. 3293–3296.
Caridi, M. , and S. Cavalieri . 2004. Multi-agent systems in production planning and control: An overview. Production Planning & Control 15 (2):106–118.
Carpenter, G. A. , and S. Grossberg . 1987. ART2: Self-organization of stable category recognition codes for analog input patterns. Applied Optics 26:4919–4930.
Cernanský, M. , M. Makula , and L. Benusková . 2007. Organization of the state space of a simple recurrent network before and after training on recursive linguistic structures. Neural Networks 20 (2):236–244.
Chakravarthy, V. 2007. Info cardio advanced medical image analysis using GIIM, IISM with CBR & RBR intelligence. Proceedings of World Academy of Science, Engineering and Technology 33:315–319.
Chalup, S. K. , and A. D. Blair . 2003. Incremental training of first order recurrent neural networks to predict a context-sensitive language. Neural Networks 16 (7):955–972.

Chandraker, R. , A. A. West , and D. J. Williams . 1990. Intelligent control of adhesive dispensing. Int. J. Computer Integrated Manufacturing 3 (1):24–34.
Charniak, E. , and D. McDermott . 1985. Introduction to Artificial Intelligence. Addison-Wesley, Reading, MA.
Cheetham, W. 2005. Tenth anniversary of plastics color matching. Artificial Intelligence Magazine 26 (3):51–61.
Cheetham, W. , and K. Goebel . 2007. Appliance call center: A successful mixed-initiative case study. AI Magazine 28 (2):89–100.
Cherian, R. P. , L. N. Smith , and P. S. Midha . 2000. A neural network approach for selection of powder metallurgy materials and process parameters. Artificial Intelligence in Engineering 14:39–44.
Chow, M. Y. , S. Altug , and H. J. Trussell . 1999. Heuristic constraints enforcement for training of and knowledge extraction from a fuzzy/neural architecture – Part I: Foundation. IEEE Transactions on Fuzzy Systems 7 (2):143–150.
Choy, K. W. , A. A. Hopgood , L. Nolle , and B. C. O'Neill . 2003. Design and implementation of an inter-process communication model for an embedded distributed processing network. In International Conference on Software Engineering Research and Practice (SERP'03). Las Vegas, NV. 239–245.
Coad, P. , and E. Yourdon . 1990. OOA: Object-oriented analysis. Prentice Hall, Englewood Cliffs, NJ.
Collinot, A. , C. Le Pape , and G. Pinoteau . 1988. SONIA: A knowledge-based scheduling system. Artificial Intelligence in Engineering 3:86–94.
Cook, R. D. , D. S. Malkus , M. E. Plesha , and R. J. Witt . 2001. Concepts and Applications of Finite Element Analysis. 4th ed. John Wiley & Sons, New York, NY.
Cunningham, P . 1998. A case study on the use of model-based systems for electronic fault diagnosis. Artificial Intelligence in Engineering 12:283–295.
Cvijovic, D. , and J. Klinowski . 1995. Taboo search – An approach to the multiple minima problem. Science 267:664–666.
Dague, P. , O. Jehl , P. Deves , P. Luciani , and P. Taillibert . 1991. When oscillators stop oscillating. In International Joint Conference on Artificial Intelligence (IJCAI'91). Sydney, Australia. 1109–1115.
Dague, P. , P. Deves , Z. Zein , and J. P. Adam . 1987. DEDALE: An expert system in VM/Prolog. In Knowledge-based expert systems in industry, edited by J. Kriz . Ellis Horwood, Chichester, UK. 57–68.
Dantzig, G. B. 1998. Linear Programming and Extensions. Princeton University Press, Princeton, NJ.
Dash, E. 1990. Diagnosing furnace problems with an expert system. SPIE-Applications of Artificial Intelligence VIII 1293:966–971.
Dattero, R. , J. J. Kanet , and E. M. White . 1989. Enhancing manufacturing planning and control systems with artificial intelligence techniques. In Knowledge-based Systems in Manufacturing, edited by A. Kusiak . Taylor and Francis, Philadelphia, PA. 137–150.
Davidson, E. M. , S. D. J. McArthur , and J. R. McDonald . 2003. A toolset for applying model-based reasoning techniques to diagnostics for power systems protection. IEEE Transactions on Power Systems 18 (2):680–687.
Davis, L. , and S. Coombs . 1987. Optimizing network link sizes with genetic algorithms. In Modelling and Simulation Methodology: Knowledge systems paradigms, edited by M. S. Elzas , T. I. Oren and B. P. Zeigler . North Holland, Amsterdam, Netherlands.
Davis, L. , ed. 1991. Handbook of Genetic Algorithms. Van Nostrand Reinhold, New York, NY.
de Castro, L. N. , and J. I. Timmis . 2003. Artificial immune systems as a novel soft computing paradigm. Soft Computing 7:526–544.
de Koning, K. , B. Bredeweg , J. Breuker , and B Wielinga . 2000. Model-based reasoning about learner behaviour. Artificial Intelligence 117 (2):173–229.
de Mantaras, R.L. , D. McSherry , D. Bridge , D. Leake , B. Smyth , S. Craw , B. Faltings , M-L. Maher , M. T. Cox , K. Forbus , M. Keane , A. Aamody , and I. Watson 2006. Retrieval, reuse, revision, and retention in case-based reasoning. Knowledge Engineering Review 20 (3):215–240.
DeKleer, J . 1986a. An assumption-based TMS. Artificial Intelligence 28:127–162.

DeKleer, J . 1986b. Extending the ATMS. Artificial Intelligence 28:163–196.
DeKleer, J . 1986c. Problem-solving with the ATMS. Artificial Intelligence 28:197–224.
Dekleer, J. , and B. C. Williams . 1987. Diagnosing multiple faults. Artificial Intelligence 32:97–130.
Demaid, A. , and J. Zucker . 1988. A conceptual model for materials selection. Metals and Materials, May, 291–297.
Demaid, A. , and J. Zucker . 1992. Prototype-oriented representation of engineering design knowledge. Artificial Intelligence in Engineering 7:47–61.
Dietterich, T. G. , and D. G. Ullman . 1987. FORLOG: A logic-based architecture for design. In Expert Systems in Computer-aided Design, edited by J. Gero . Elsevier, Amsterdam, Netherlands, 1–17.
Dignum, V . 2018. Ethics in artificial intelligence: Introduction to the special issue. Ethics and Information Technology 20 (1).
Diniz, P. S. R. , E. A. B. da Silva , and S. L. Netto . 2010. Digital Signal Processing: System Analysis and Design. 2nd ed. Cambridge University Press, Cambridge, UK.
Dorigo, M . 2001. Ant algorithms solve difficult optimization problems. Lecture Notes in Artificial Intelligence 2159:11–22.
Dorigo, M. , and T. Stützle . 2004. Ant Colony Optimization. MIT Press, Cambridge, MA.
Duda, R. O. , J. G. Gashnig , and P. E. Hart . 1979. Model design in the PROSPECTOR consultant system for mineral exploration. In Expert Systems in the Micro-electronic Age, edited by D. Michie . Edinburgh University Press, Edinburgh, UK.
Duda, R. O. , and P. E. Hart . 1972. Use of the Hough transform to detect lines and curves in pictures. Communications of the ACM 15 (1):11–15.
Duda, R. O. , and P. E. Hart . 1973. Pattern Classification and Scene Analysis. Wiley, New York, NY.
Duda, R. O. , P. E. Hart , and N. J. Nilsson . 1976. Subjective Bayesian methods for rule-based inference systems. In National Computer Conference. AFIPS. 1075–1082.
Durfee, E. H. , V. R. Lesser , and D. D. Corkhill . 1985. Increasing coherence in a distributed problem solving network. In 9th International Joint Conference on Artificial Intelligence (IJCAI'85). Los Angeles, CA. 1025–1030.
Dyer, M. G. , M. Flowers , and J. Hodges . 1986. EDISON: An engineering design invention system operating naively. Artificial Intelligence in Engineering 1:36–44.
Elman, J. L. 1991. Distributed representations, simple recurrent networks, and grammatical structure. Machine Learning 7 (2–3):195–225.
El-Mihoub, T. , A. A. Hopgood , and L. Nolle . 2020. Self-adaptive learning for hybrid genetic algorithms. Evolutionary Intelligence.
El-Mihoub, T. , A. A. Hopgood , L. Nolle , and A. Battersby . 2004. Performance of hybrid genetic algorithms incorporating local search. In 18th European Simulation Multiconference (ESM2004), edited by G. Horton . Magdeburg, Germany. 154–160.
El-Mihoub, T. , A. A. Hopgood , L. Nolle , and A. Battersby . 2006a. Hybrid genetic algorithms – A review. Engineering Letters 13:124–137.
El-Mihoub, T. , A. A. Hopgood , L. Nolle , and A. Battersby . 2006b. Self-adaptive Baldwinian search in hybrid genetic algorithms. In Computational Intelligence: Theory and Applications – Advances in Soft Computing, vol. 38, edited by B. Reusch . Springer, Berlin, Germany. 597–602.
Eriksson, H. 1996. Expert systems as knowledge servers. IEEE Intelligent Systems 11 (3):14–19.
Erman, L. D. , F. Hayes-Roth , V. R. Lesser , and D. R. Reddy . 1980. The Hearsay-II speech understanding system: Integrating knowledge to resolve uncertainty. ACM Computing Surveys 12 (2):213–253.
Feigenbaum, E. A. . 1988. Blackboard systems, edited by R. S. Englemore and A. J. Morgan . Addison-Wesley, Reading, MA.
Fenton, W. G. , T. M. McGinnity , and L. P. Maguire . 2001. Fault diagnosis of electronic systems using intelligent techniques: A review. IEEE Transactions on Systems Man and Cybernetics Part C – Applications and Reviews 31 (3):269–281.
Ferguso, A. , and D. Bridge . 2000. Options for query revision when interacting with case retrieval systems. Expert Update 3 (1):16–27.

Ferguson, I. A. 1995. Integrated control and coordinated behaviour: A case for agent models. In Intelligent Agents: Theories, Architectures and Languages, edited by M. Wooldridge and N. R. Jennings . Springer-Verlag, Berlin, Germany. 203–218.
Fernandez, S. , R. Aler , and D. Borrajo . 2005. Machine learning in hybrid hierarchical and partial-order planners for manufacturing domains. Applied Artificial Intelligence 19 (8):783–809.
Fielden, G. B. R. 1963. Engineering design. HMSO, London, UK.
Fikes, R. E. , and N. J. Nilsson . 1971. STRIPS: A new approach to the application of theorem proving to problem solving. Artificial Intelligence 2:189–208.
Fikes, R. E. , P. E. Hart , and N. J. Nilsson . 1972. Learning and executing generalized robot plans. Artificial Intelligence 3:251–288.
Finin, T. , Y. Labrou , and J. Mayfield . 1997. KQML as an agent communication language. In Software Agents, edited by J. M. Bradshaw . MIT Press, Cambridge, MA. 291–316.
Fink, P. K. , and J. C. Lusth . 1987. Expert systems and diagnostic expertise in the mechanical and electrical domains. IEEE Transactions on Systems, Man, and Cybernetics 17 (3):340–349.
Firth, N . 2007. Stepping up a gear: The latest version of Honda's Asimo boasts a host of features that takes the world's most advanced robot closer to full autonomy. Engineer 293:26–27.
Forgy, C. L. 1982. Rete: A fast algorithm for the many pattern/many object pattern match problem. Artificial Intelligence 19 (1):17–37.
Frank, S. L. 2006. Learn more by training less: Systematicity in sentence processing by recurrent networks. Connection Science 18 (3):287–302.
Franklin, G. F. , J. D. Powell , and A. Emami-Naeini . 2020. Feedback Control of Dynamic Systems. 8th ed. Pearson Education, Upper Saddle River, NJ.
Fulton, S. L. , and C. O. Pepe . 1990. An introduction to model-based reasoning. AI Expert 5 (1):48–55.
Galindo, C. , J. A. Fernandez-Madrigal , and J. Gonzalez . 2004. Improving efficiency in mobile robot task planning through world abstraction. IEEE Transactions on Robotics and Automation 20 (4):677–690.
Galuszka, A. , and A. Swierniak . 2010. Planning in multi-agent environment using STRIPS representation and non-cooperative equilibrium strategy. Journal of Intelligent & Robotic Systems 58 (3–4):239–251.
Garrett, C. R. , T. Lozano-Pérez , and L. P. Kaelbling . 2017. STRIPS planning in infinite domains. Preprint arXiv:1701.00287.
Gers, F. A. , J. Schmidhuber , and F. Cummins . 2000. Learning to forget: Continual prediction with LSTM. Neural Computation 12 (10): 2451–2471.
Ghosh-Dastidar, S. , and H. Adeli . 2009. Spiking neural networks. International Journal of Neural Systems 19 (4):295–308.
Glover, F. , and M. Laguna . 1997. Tabu Search. Kluwer, Norwell, MA.
Gogos, C. , P. Alefragis , and E. Housos . 2005. Public enterprise water pump scheduling system. In 10th IEEE Conference on Emerging Technologies and Factory Automation.
Goldberg, D. E. 1989. Genetic Algorithms in Search, Optimization, and Machine Learning. Addison Wesley, Reading, MA.
Goldberg, D. E. , B. Korb , and K. Deb . 1989. Messy genetic algorithms: Motivation, analysis, and first results. Complex Systems 3:493–530.
Goodfellow, I. J. , J. Pouget-Abadie , M. Mirza , B. Xu , D. Warde-Farley , S. Ozair , A. Courville , and Y. Bengio . 2014. Generative adversarial nets. In Advances in Neural Information Processing Systems (NIPS'14), edited by Z. Ghahramani , M. Welling , C. Cortes , N. D. Lawrence , and K. Q. Weinberger . Curran Associates, Red Hook, NY. 3:2672–2680.
Gorman, B. , and M. Humphrys . 2007. Imitative learning of combat behaviours in first-person computer games. In Proceedings of 10th International Conference on Computer Games (CGAMES07). Louisville, KY. 85–90.
Gouaillier, D. , V. Hugel , P. Blazevic , C. Kilner , J. Monceaux , P. Lafourcade , B. Marnier , J. Serre , and B. Maisonnier . 2009. Mechatronic design of NAO humanoid. In IEEE Int. Conf. on Robotics and Automation. Kobe, Japan.
Gray, F. 1953. Pulse code communication. U.S. Patent 2,632,058.
Greenberg, J. , and C. N. Reid . 1981. A simple design task (with the aid of a microcomputer). In 2nd International Conference on Engineering Software. Southampton, UK. 926–942.

Greff, K. , R. K. Srivastava , J. Koutnik , B. R. Steunebrink , and J. Schmidhuber . 2017. LSTM: A search space odyssey. IEEE Transactions on Neural Networks and Learning Systems 28 (10): 2222–2232.
Griewank, A. O. 1981. Generalized descent for global optimization. Journal of Optimization Theory and Applications 34:11–39.
Güera, D. and E. J. Delp . 2018. Deepfake video detection using recurrent neural networks. In 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), Auckland, New Zealand. 1–6.
Hallam, N. J. , A. A. Hopgood , and N. Woodcock . 1990. Defect classification in welds using a feedforward network within a blackboard system. In International Neural Network Conference (INNC'90). Paris, France. 353–356.
Halmshaw, R. 1987. Non-destructive Testing. Edward Arnold, London, UK.
Harel, D . 1988. On visual formalisms. Communications of the ACM 31 (5):514–530.
Hart, P. E. , R. O. Duda , and M. T. Einaudi . 1978. PROSPECTOR: A computer-based consultation system for mineral exploration. Math Geology 10 (5):589–610.
Hausen, R. and B. E. Robertson . 2020. Morpheus: A deep learning framework for the pixel-level analysis of astronomical image data. The Astrophysical Journal Supplement Series 248(1).
Hecht-Nielson, R . 1990. Neurocomputing. Addison-Wesley, Reading, MA.
Hendler, J. , A. Tate , and M. Drummond . 1990. AI planning: Systems and techniques. AI Magazine 11 (2):61–77.
Herrera F. and M. Lozano . 1996. Adaptation of genetic algorithm parameters based on fuzzy logic controllers. In Genetic Algorithms and Soft Computing, edited by F. Herrera and J. L. Verde-Gay , Physica-Verlag, Heidelberg, Germany. 95–125.
Hinton, G. E. and R. R. Salakhutdinov . 2006. Reducing the dimensionality of data with neural networks. Science, 313 (5786):504–507.
Hirschman, I. I. , and D. V. Widder . 2005. The Convolution Transform. Dover Publications, Mineola, NY.
Hochreiter, S. , and J. Schmidhuber . 1997. Long short-term memory. Neural Computation 9 (8): 1735–1780.
Holland, J. H. 1975. Adaptation in Natural and Artificial Systems. University of Michigan Press, Ann Arbor, MI.
Holland, J. H. , and J. S. Reitman . 1978. Cognitive systems based on adaptive algorithms. In Pattern-directed Inference Systems, edited by D. A. Waterman and F. Hayes-Roth . Academic Press, New York, NY. 313–329.
Holmes, N . 2003. Artificial intelligence: Arrogance or ignorance? IEEE Computer 36 (11): 118–120.
Hopfield, J. J. 1982. Neural networks and physical systems with emergent collective computational abilities. Proc. National Academy of Science. USA. 2554–2558.
Hopgood, A. A. 1989. An inference mechanism for selection, and its application to polymers. Artificial Intelligence in Engineering 4:197–203.
Hopgood, A. A. 1994. Rule-based control of a telecommunications network using the blackboard model. Artificial Intelligence in Engineering 9 (1): 29–38.
Hopgood, A. A. 2003. Artificial intelligence: Hype or reality? IEEE Computer 36 (5): 24–28.
Hopgood, A. A. 2005. The state of artificial intelligence. In Advances in Computers, edited by M. V. Zelkowitz . Elsevier, Oxford, UK. 65:1–75.
Hopgood, A. A. , and A. J. Hopson . 1991. The common traffic model: A universal model for communications networks. In Institution of Radio and Electronic Engineers Conference (IREECON'91). Sydney, Australia. 61–64.
Hopgood, A. A. , and A. Mierzejewska . 2008. Transform ranking: A new method of fitness scaling in genetic algorithms. In Research and Development in Intelligent Systems XXV, edited by M. Bramer , F. Coenen and M. Petridis . Springer, Berlin, Germany. 349–354.
Hopgood, A. A. , H. J. Phillips , P. D. Picton , and N. St. J. Braithwaite . 1998. Fuzzy logic in a blackboard system for controlling plasma deposition processes. Artificial Intelligence in Engineering 12:253–260.
Hopgood, A. A. , N. Woodcock , N. J. Hallam , and P. D. Picton . 1993. Interpreting ultrasonic images using rules, algorithms and neural networks. European Journal of Nondestructive Testing 2:135–149.

Hornick, K. , M. Stinchcombe , and H. White . 1989. Multilayer feedforward networks are universal approximators. Neural Networks 2 (5):359–366.
Hossack, J. , S. D. J. McArthur , E. Davidson , J. R. McDonald , and T. Cumming . 2002. A multi-agent intelligent interpretation system for power system disturbance diagnosis. In Applications and Innovations in Intelligent Systems X, edited by A. L. Macintosh , R. Ellis and F. Coenen . Springer-Verlag, London, UK. 91–104.
Howe, A. E. , P. R. Cohen , J. R. Dixon , and M. K. Simmons . 1986. DOMINIC: A domain-independent program for mechanical engineering design. Artificial Intelligence in Engineering 1:23–28.
Hsu, E. , and D. L. McGuinness . 2003. KSL wine agent: A semantic web testbed application. In International Workshop on Description Logics (DL2003). Rome, Italy.
Huang, J. K. , M. T. Ho , and R. L. Ash . 1992. Expert systems for automated maintenance of a Mars oxygen production system. Journal of Spacecraft and Rockets 29:425–431.
Huang, J. T. , and Y. S. Liao . 2000. A wire-EDM maintenance and fault-diagnosis expert system integrated with an artificial neural network. International Journal of Production Research 38:1071–1082.
Hunt, J . 1997. Case-based diagnosis and repair of software faults. Expert Systems 14:15–23.
Jahan, A. , M. Y. Ismail , S. M. Sapuan , and F. Mustapha . 2010. Material screening and choosing methods – A review. Materials & Design 31 (2):696–705.
Jang, J.-S. R. 1993. ANFIS: Adaptive-network-based fuzzy inference system. IEEE Transactions on Systems, Man, and Cybernetics 23 (03):665–685.
Jennings, A. J. 1989. Artificial intelligence: A tool for productivity. In Institution of Engineers (Australia) National Conference. Perth, Australia.
Jennings, N. R. 2000. On agent-based software engineering. Artificial Intelligence 117:277–296.
John, R. , and S. Coupland . 2007. Type-2 fuzzy logic: A historical view. IEEE Computational Intelligence Magazine 2 (1):57–62.
Johnson, J. H. , and P. D. Picton . 1995. Concepts in Artificial Intelligence. Butterworth-Heinemann, Oxford, UK.
Johnson, J. H. , N. J. Hallam , and P. D. Picton . 1990. Safety critical neurocomputing: Explanation and verification in knowledge augmented neural networks. In IEE Colloquium on Human–Computer Interaction. London, UK.
Jozefowicz, R. , W. Zaremba , and I. Sutskever . 2015. An empirical exploration of recurrent network architectures. In 32nd International Conference on Machine Learning, ICML 2015, 3:2332–2340.
Kang, C. W. , and M. W. Golay . 1999. A Bayesian belief network-based advisory system for operational availability focused diagnosis of complex nuclear power systems. Expert Systems with Applications 17:21–32.
Karr, C. L. 1991a. Applying genetics to fuzzy logic. AI Expert 6 (3):38–43.
Karr, C. L. 1991b. Genetic algorithms for fuzzy controllers. AI Expert 6 (2):26–33.
Kavakli, M . 2001. NoDes: kNOwledge-based modeling for detailed DESign process – From analysis to implementation. Automation in Construction 10:399–416.
Khmeleva, E. , A. A. Hopgood , L. Tipi , and M. Shahidan . 2018. Fuzzy-logic controlled genetic algorithm for the rail-freight crew-scheduling problem. KI – Künstliche Intelligenz 32 (1):61–75.
Kim, K. H. , and J. K. Park . 1993. Application of hierarchical neural networks to fault diagnosis of power systems. International Journal of Electrical Power and Energy Systems 15 (2):65–70.
Kim, N. H. , B. V. Sankar , and A. V. Kumar . 2018. Introduction to Finite Element Analysis and Design. 2nd ed. John Wiley & Sons, New York, NY.
Kim, S. H. , and N. P. Suh . 1989. Formalizing decision rules for engineering design. In Knowledge-based Systems in Manufacturing, edited by A. Kusiak . Taylor and Francis, Philadelphia, PA. 33–44.
Kinny, D. , and M. Georgeff . 1991. Commitment and effectiveness in situated agents. In 12th International Joint Conference on Artificial Intelligence (IJCAI'91). Sydney, Australia. 82–88.
Kirkpatrick, S. , C. D. Gelatt , and M. P. Vecchi . 1983. Optimization by simulated annealing: Quantitative studies. Science 220:671–680.
Klarreich, E . 2002. Inspired by immunity. Nature 415:468–470.
Knight, K . 1990. Connectionist ideas and algorithms. Communications of the ACM 33 (11):59–74.

Kohonen, T . 1987. Adaptive, associative, and self-organizing functions in neural computing. Applied Optics 26 (23):4910–4918.
Kohonen, T . 1988. Self-organization and Associative Memory. 2nd ed. Springer-Verlag, Berlin, Germany.
Komzsik, L. 2009. What Every Engineer Should Know About Computational Techniques of Finite Element Analysis. 2nd ed. CRC Press, Boca Raton, FL.
Koza, J. R. , I. I. I. Bennett , H. Forrest , D. Andre , and M. A. Keane (eds.) 1999. Genetic Programming III: Darwinian invention and problem solving. Morgan Kaufmann, San Francisco, CA.
Kreinovich, V. , C. Quintana , and O. Fuentes . 1993. Genetic algorithms: What fitness scaling is optimal? Cybernetics and Systems: An International Journal 24:9–26.
Krizhevsky, A. , I. Sutskever , and G. E. Hinton . 2017. ImageNet classification with deep convolutional neural networks. Communications of the ACM 60 (6):84–90.
Kumar, M. , N. Stoll , D. Kaber , K. Thurow , and R. Stoll . 2007. Fuzzy filtering for an intelligent interpretation of medical data. IEEE International Conference on Automation Science and Engineering. Scottsdale, AZ. 225–230
Laffey, T. J. , P. A. Cox , J. L. Schmidt , S. M. Kao , and J. Y. Read . 1988. Real-time knowledge-based systems. AI Magazine 9 (1):27–45.
Laird, J. E. , A. Newell , and P. S. Rosenbloom . 1987. SOAR: An architecture for general intelligence. Artificial Intelligence 33:1–64.
Laird, J. E. , P. S. Rosenbloom , and A. Newell . 1986. Chunking in SOAR: The anatomy of a general learning mechanism. Machine Learning 1:11–46.
Lamarck, J.-B. 1809. Philosophie Zoologique. Paris, France.
LeCun, Y. , L. Bottou , Y. Bengio , and P. Haffner . 1998. Gradient-based learning applied to document recognition. Proceedings of the IEEE 86 (11):2278–2323.
Lee, C. C. 1990a. Fuzzy logic in control systems: Fuzzy logic controller – Part I. IEEE Transactions on Systems, Man and Cybernetics 20 (2):404–418.
Lee, C. C. 1990b. Fuzzy logic in control systems: Fuzzy logic controller – Part II. IEEE Transactions on Systems, Man and Cybernetics 20 (2):419–435.
Lee, S. , and H. Rowlands , H. 2005. Finding robust optima with a diploid genetic algorithm. International Journal of Simulation: Systems, Science and Technology 6 (9):73–79.
Leitch, R. , R. Kraft , and R. Luntz . 1991. RESCU: A real-time knowledge based system for process control. IEE Proceedings-D 138 (3):217–227.
Lesser, V. R. , J. Pavlin , and E. Durfee . 1988. Approximate processing in real-time problem solving. AI Magazine 9 (1):49–61.
Lesser, V. R. , R. D. Fennell , L. D. Erman , and D. R. Reddy . 1975. Organization of the Hearsay-II speech understanding system. IEEE Transactions on Acoustics, Speech, and Signal Processing 23:11–23.
Li, G. , A. A. Hopgood , and M. J. Weller . 2003. Shifting matrix management: A model for multi-agent cooperation. Engineering Applications of Artificial Intelligence 16:191–201.
Lippmann, R. P. 1987. An introduction to computing with neural nets. IEEE ASSP Magazine 4 (2):4–22.
Lirov, Y . 1990. Systematic invention for knowledge engineering. AI Expert 5 (7):28–33.
Liu, B . 1988. Scheduling via reinforcement. Artificial Intelligence in Engineering 3:76–85.
Lu, Y. , T. Q. Chen , and B. Hamilton . 1998. A fuzzy diagnostic model and its application in automotive engineering diagnosis. Applied Intelligence 9:231–243.
Lumer, E. D. , and B. Faieta . 1994. Diversity and adaptation in populations of clustering ants. In Proc. 3rd Int. Conf. on Simulation of Adaptive Behavior: From animal to animats, edited by D. Cliff , P. Husbands , J. Meyer and S. Wilson . MIT Press/Bradford Books, Cambridge, MA.
Maass, W . 1997. Networks of spiking neurons: The third generation of spiking neural network models. Neural Networks 10 (9):1659–1671.
Maderlechner, G. , E. Egeli , and F. Klein . 1987. Model guided interpretation based on structurally related image primitives. In Knowledge-based expert systems in industry, edited by J. Kriz . Ellis Horwood, Chichester, UK. 91–97.
Mamdani, E. H. 1977. Application of fuzzy logic to approximate reasoning using linguistic synthesis. IEEE Transactions on Computers 26 (12):1182–1191.
Manoj, C. , and N. Nagarajan . 2003. The application of artificial neural networks to magnetotelluric time-series analysis. Geophysical Journal International 153 (2):409–423.

Marefat, M. , and J. Britanik . 1997. Case-based process planning using an object-oriented model representation. Robotics and Computer Integrated Manufacturing 13 (3):229–251.
Mateis, C. , M. Stumptner , and F. Wotawa . 2000. Locating bugs in Java programs – First results of the Java diagnosis experiments project. Lecture Notes in Artificial Intelligence 1821:174–183.
Matsui, Y. , M. Kanoh , S. Kato , and H. Itoh . 2008. Generating interactive facial expression of communication robots using simple recurrent network. In PRICAI 2008: Trends in Artificial Intelligence, edited by T. B. Ho and Z. H. Zhou . 1016–1021.
Matsui, Y. , M. Kanoh , S. Kato , T. Nakamura , and H. Itoh . 2009. Evaluating a model for generating interactive facial expressions using simple recurrent network. In SMC 2009: IEEE International Conference on Systems, Man and Cybernetics. 1639–1644.
McCormick, G. , and R. S. Powell . 2004. Derivation of near-optimal pump schedules for water distribution by simulated annealing. Journal of the Operational Research Society 55 (7):728–736.
McCulloch, W. S. , and W. Pitts . 1943. A logical calculus of the ideas immanent in nervous activity. The Bulletin of Mathematical Biophysics 5 (4): 115–133.
McQueen, T. , A. A. Hopgood , T. J. Allen , and J. A. Tepper . 2005. Extracting finite structure from infinite language. Knowledge-Based Systems 18 (4–5):135–141.
Melioli, G. , C. Spenser , G. Reggiardo , G. Passalacqua , E. Compalati , A Rogkakou , A. M. Riccio , E. Di Leo , E. Nettis , and G. W. Canonica . 2014. Allergenius, an expert system for the interpretation of allergen microarray results. The World Allergy Organization Journal 7 (1):15.
Mendel, J. M. 2007. Type-2 fuzzy sets and systems: An overview. IEEE Computational Intelligence Magazine 2 (1):20–29.
Mettrey, W . 1991. A comparative evaluation of expert system tools. IEEE Computer 24 (2):19–31.
Micalizio, R. 2009. A distributed control loop for autonomous recovery in a multi-agent plan. In 21st International Joint Conference on Artificial Intelligence (IJCAI'09). Pasadena, USA. 1760–1765.
Michie, D. , and R. A. Chambers . 1968. BOXES: An experiment in adaptive control. In Machine Intelligence 2, edited by E. Dale and D. Michie . Oliver and Boyd, Edinburgh, UK. 137–152.
Milne, R . 1987. Strategies for diagnosis. IEEE Transactions on Systems, Man, and Cybernetics 17 (3):333–339.
Minton, S. , J. G. Carbonell , C. A. Knoblock , D. R. Kuokka , O. Etzioni , and Y. Gil . 1989. Explanation-based learning: A problem-solving perspective. Artificial Intelligence 40:63–118.
Mintzberg, H. 1979. The Structuring of Organizations. Prentice-Hall, Englewood Cliffs, NJ.
Mitchell, M. 1996. An Introduction to Genetic Algorithms. MIT Press, Cambridge, MA.
Mitchell, T. M. , P. E. Utgoff , and R. Banerji . 1983. Learning by experimentation: Acquiring and refining problem-solving heuristics. In Machine learning: An artificial intelligence approach, vol. 1, edited by R. Michalski , J. G. Carbonell and T. M. Mitchell , 163–190.
Moallem, P. , and S. A. Monadjemi . 2010. An efficient MLP-learning algorithm using parallel tangent gradient and improved adaptive learning rates. Connection Science 22 (4):373–392.
Montani, S. , P. Magni , R. Bellazzi , C. Larizza , A. V. Roudsari , and E. R. Carson . 2003. Integrating model-based decision support in a multi-modal reasoning system for managing type 1 diabetic patients. Artificial Intelligence in Medicine 29 (1–2):131–151.
Moody, S . 1980. The role of industrial design in technological innovation. Design Studies 1 (6):329–339.
Moskewicz, M. W. , C. F. Madigan , Y. Zhao , L. T. Zhang , and S. Malik . 2001. Chaff: Engineering an efficient SAT solver. In 38th Design Automation Conference Proceedings. ACM, New York, NY. 530–535.
Motta, E. , M. Eisenstadt , K. Pitman , and M. West . 1988. Support for knowledge acquisition in the Knowledge Engineer's Assistant (KEATS). Expert Systems 5 (1):6–28.
Mouelhi, O. , P. Couturier , and T. Redarce . 2009. Multicriteria optimization and evaluation in complex products design. In 2009 International Conference on Computers and Industrial Engineering. IEEE, New York, NY. 590–595.
Muraina, I. O. , M. A. Rahman , and I. A. Adeleke . 2016. Statistical approaches and decision making towards bivariate and multivariate analyses with Visirule. British Journal of Education, Society & Behavioural Science 14 (2): 1–10.

Murthy, S. S. , and S. Addanki . 1987. PROMPT: An innovative design tool. In Expert Systems in Computer-Aided Design, edited by J. Gero . North-Holland, Amsterdam, Netherlands. 323–341.

Mustonen, V. , and M. Lässig . 2009. From fitness landscapes to seascapes: Non-equilibrium dynamics of selection and adaptation. Trends in Genetics 25 (3):111–119.

Mustonen, V. , and M. Lässig . 2010. Fitness flux and ubiquity of adaptive evolution. Proceedings of the National Academy of Sciences 107 (9):4248–4253.

Mutawa, A. M. , and M. A. Alzuwawi . 2019. Multilayered rule-based expert system for diagnosing uveitis. Artificial Intelligence in Medicine 99:101691.

Navichandra, D. , and D. H. Marks . 1987. Design exploration through constraint relaxation. In Expert Systems in Computer-aided Design, edited by J. Gero . Elsevier, Amsterdam, Netherlands. 481–509.

Netten, B. D. 1998. Representation of failure context for diagnosis of technical applications. Advances in Case-based Reasoning 1488:239–250.

Newell, A . 1982. The knowledge level. Artificial Intelligence 18 (1):87–127.

Niemüller, T. , A. Ferrein , and G. Lakemeyer . 2010. A Lua-based behavior engine for controlling the humanoid robot Nao. In RoboCup 2009: Robot Soccer World Cup XIII, edited by J. Baltes , M. Lagoudakis , T. Naruse , S. Ghidary . Lecture Notes in Computer Science, vol. 5949. Springer, Berlin, Germany. 240–251.

Nii, H. P. 1986. Blackboard systems, part one: The blackboard model of problem solving and the evolution of blackboard architectures. AI Magazine 7:38–53.

Nolle, L. , D. A. Armstrong , A. A. Hopgood , and A. Ware . 2002a. Simulated annealing and genetic algorithms applied to finishing mill optimisation for hot rolling of wide steel strip. Int. Journal of Knowledge-Based Intelligent Engineering Systems 6:104–110.

Nolle, L. , A. Goodyear , A. A. Hopgood , P. D. Picton , and N. S. Braithwaite . 2002b. Automated control of an actively compensated Langmuir probe system using simulated annealing. Knowledge-Based Systems 15:349–354.

Nolle, L. , K. C. P. Wong , and A. A. Hopgood . 2002c. DARBS: A distributed blackboard system. In Research and Development in Intelligent Systems XVIII, edited by M. Bramer , F. Coenen and A. Preece . Springer-Verlag, London, UK. 161–170.

Nolle, L. , A. Goodyear , A. A. Hopgood , P. D. Picton , and N. S. Braithwaite . 2005. Improved simulated annealing with step width adaptation for Langmuir probe tuning. Engineering Optimization 37:463–477.

Nolle, L. , D. A. Armstrong , A. A. Hopgood , and J. A. Ware . 1999. Optimum work roll profile selection in the hot rolling of wide steel strip using computational intelligence. Lecture Notes in Computer Science 1625:435–452.

O'Leary, D. E. 1998. Knowledge acquisition from multiple experts: An empirical study. Management Science 44 (8):1049–1058.

Oh, K. W. , and C. Quek . 2001. BBIPS: A blackboard-based integrated process supervision. Engineering Applications of Artificial Intelligence 14 (6):703–714.

Pascoe, G. A. 1986. Elements of object-oriented programming. Byte 11 (8):139–144.

Passow, B. N. , M. A. Gongora , S. Coupland , and A. A. Hopgood . 2008. Real-time evolution of an embedded controller for an autonomous helicopter. In Proc. 2008 IEEE Congress on Evolutionary Computation (CEC 2008). Hong Kong.

Pedersen, G. K. M. , and D. E. Goldberg . 2004. Dynamic uniform scaling for multiobjective genetic algorithms. Lecture Notes in Computer Science 3103:11–23.

Penrose, R . 1999. The Emperor's New Mind. Oxford University Press, Oxford, UK.

Philipp, A. , P. M. Della-Marta , J. Jacobeit , D. R. Fereday , P. D. Jones , A. Moberg , and H. Wanner . 2007. Long-term variability of daily North Atlantic-European pressure patterns since 1850 classified by simulated annealing clustering. Journal of Climate 20 (16):4065–4095.

Picton, P. D. , J. H. Johnson , and N. J. Hallam . 1991. Neural networks in safety-critical systems. In 3rd International Congress on Condition Monitoring and Diagnostic Engineering Management. Southampton, UK.

Pons, D. J. , and J. K. Raine . 2005. Design mechanisms and constraints. Research in Engineering Design 16 (1–2):73–85.

Portinale, L. , D. Magro , and P. Torasso . 2004. Multi-modal diagnosis combining case-based and model-based reasoning: A formal and experimental analysis. Artificial Intelligence 158 (2):109–153.

Poslad, S. and P. Charlton . 2001. Standardizing agent interoperability: The FIPA approach. In Multi-Agent Systems and Applications, edited by M. Luck , V. Mařík , O. Štěpánková , and R. Trappl . Lecture Notes in Computer Science, vol. 2086. Springer, Berlin, Germany. 98–117.

Prang, J. , H. D. Huemmer , and W. Geisselhardt . 1996. A system for simulation, monitoring and diagnosis of controlled continuous processes with slow dynamics. Knowledge-based Systems 9:525–530.

Price, C. J. 1998. Function-directed electrical design analysis. Artificial Intelligence in Engineering 12:445–456.

Price, C. J. , and J. E. Hunt . 1991. Automating FMEA through multiple models. In Research and development in expert systems VIII, edited by I. Graham and R. Milne . Cambridge University Press, Cambridge, UK, 25–39.

Price, C. J. , and J. Hunt . 1989. Simulating mechanical devices. In Pop-11 Comes of Age: The advancement of an AI programming language, edited by J. A. D. W. Anderson . Ellis Horwood, Chichester, UK, 217–237.

Quinlan, J. R. 1983. Inferno: A cautious approach to uncertain inference. The Computer Journal 26 (2):255–269.

Ramaswamy, S. , T. J. Cutright , and H. K. Qammar . 2005. Control of a continuous bioreactor using model predictive control. Process Biochemistry 40 (8):2763–2770.

Ramos, V. , and F. Almeida . 2000. Artificial ant colonies in digital image habitats – A mass behaviour effect study on pattern recognition. In Proc 2nd Int. Workshop on Ant Algorithms: From ant colonies to artificial ants, edited by M. Dorigo , M. Middendorf , and T. Stüzle. Brussels , Belgium. 113–116.

Refanidis, I. , and I. Vlahavas . 2001. The GRT planning system: Backward heuristic construction in forward state-space planning. Journal of Artificial Intelligence Research 15:115–161.

Reid, C. N. , and J. Greenberg . 1980. An exercise in materials selection. Metals and Materials, July, 385–387.

Rescher, N. 1976. Plausible Reasoning. Van Gorcum, Assen, Netherlands.

Riesbeck, C. K. , and R. C. Schank . 1989. Inside Case-based Reasoning. Lawrence Erlbaum Associates, Hillsdale, NJ.

Rumbaugh, J. , I. Jacobson , and G. Booch . 2010. The Unified Modeling Language Reference Manual. 2nd ed. Addison-Wesley, Reading, MA.

Rumelhart, D. E. , G. E. Hinton , and R. J. Williams . 1986a. Learning internal representations by error propagation. In Parallel Distributed Processing: Explorations in the microstructures of cognition, edited by D. E. Rumelhart and J. L. McClelland . MIT Press, Cambridge, MA.

Rumelhart, D. E. , G. E. Hinton , and R. J. Williams . 1986b. Learning representations by back-propagating errors. Nature 323 (9):533–536.

Russell, S. , and P. Norvig . 2016. Artificial Intelligence: A Modern Approach. 3rd ed. Pearson Education, London, UK.

Sacerdoti, E. D. 1974. Planning in a hierarchy of abstraction spaces. Artificial Intelligence 5:115–135.

Sadjadi, F. 2004. Comparison of fitness scaling functions in genetic algorithms with applications to optical processing. In Proc. SPIE: Optical Information Systems II, edited by B. Javidi and D. Psaltis . 356–364.

Saini, R. , N. J. Ahuja , and K. D. Bahukhandi . 2016. Fuzzy logic based advisory for handling landfill operational problems for early warning and emergency response planning, International Journal of ChemTech Research 9 (08):282–297.

Sakagami, Y. , R. Watanabe , C. Aoyama , S. Matsunaga , N. Higaki , and K. Fujimura . 2002. The intelligent ASIMO: System overview and integration. In IEEE/RSJ International Conference on Intelligent Robots and Systems. Lausanne, Switzerland.

Sammut, C. , and D. Michie . 1991. Controlling a black-box simulation of a spacecraft. AI Magazine 12 (1):56–63.

Scarl, E. A. , J. R. Jamieson , and C. I. Delaune . 1987. Diagnosis and sensor validation through knowledge of structure and function. IEEE Transactions on Systems, Man, and Cybernetics 17 (3):360–368.

Schafer, G . 1976. A Mathematical Theory of Evidence. Princeton University Press, Princeton, NJ.

Schmedding, F. , N. Sawas , and G. Lausen . 2007. Adapting the Rete-algorithm to evaluate F-logic rules. In Lecture Notes in Computer Science, 4824 LNCS:166–173. Springer Verlag, Orlando, FL.
Schwefel, H.-P. 1981. Numerical Optimization of Computer models. John Wiley & Sons, New York, NY.
Sejnowski, T. J. , and C. R. Rosenberg . 1986. NETtalk: A parallel network that learns to read aloud. Cognitive Science 14:179–211.
Shadbolt, N. , W. Hall , and T. Berners-Lee . 2006. The semantic web revisited. IEEE Intelligent Systems 21 (3):96–101.
Shen, W. M. 2002. Distributed manufacturing scheduling using intelligent agents. IEEE Intelligent Systems 17 (1):88–94.
Shen, W. M. , Q. Hao , H. J. Yoon , and D. H. Norrie . 2006. Applications of agent-based systems in intelligent manufacturing: An updated review. Advanced Engineering Informatics 20 (4):415–431.
Shieh, J-S , C-F Chou , S-J Huang , and M-C Kao . 2004. Intracranial pressure model in intensive care unit using a simple recurrent neural network through time. Neurocomputing 57:239–256.
Shortliffe, E. H. 1976. Computer-Based Medical Consultations: MYCIN. Elsevier, New York, NY.
Shortliffe, E. H. , and B. G. Buchanan . 1975. A model of inexact reasoning in medicine. Mathematical Biosciences 23:351–379.
Skyttner, L. 2005. General Systems Theory: Problems, Perspectives, Practice. 2nd ed. World Scientific, Hackensack, NJ.
Smith, R. G. , and R. Davis . 1981. Frameworks for cooperation in distributed problem solving. IEEE Transactions on Systems, Man, and Cybernetics 11 (1):61–70.
Spenser, C . 2007. Drawing on your knowledge with VisiRule. IEEE Potentials 26 (1): 20–25.
Sripada, N. R. , D. G. Fisher , and A. J. Morris . 1987. AI application for process regulation and process control. IEE Proceedings-D 134 (4):251–259.
Sriram, D. , G. Stephanopoulos , R. Logcher , D. Gossard , N. Groleau , D. Serrano , and D. Navinchandra . 1989. Knowledge-based system applications in engineering design: Research at MIT. AI Magazine 10 (3):79–96.
Stahl, B. C. 2004. Information, ethics, and computers: The problem of autonomous moral agents. Minds and Machines 14:67–83.
Steele, G. L. 1990. Common Lisp: The language. 2nd ed. Digital Press, Bedford, MA.
Steels, L . 1989. Diagnosis with a function-fault model. Applied Artificial Intelligence 3 (2–3):129–153.
Stroustrup, B . 1988. What is object-oriented programming? IEEE Software 5 (3):10–20.
Stuart, C. J. 1985. An implementation of a multi-agent plan synchronizer. In 9th International Joint Conference on Artificial Intelligence (IJCAI'85). Los Angeles, CA. 1031–1033.
Suman, B. , and P. Kumar . 2006. A survey of simulated annealing as a tool for single and multiobjective optimization. Journal of the Operational Research Society 57 (10):1143–1160.
Suman, B. , N. Hoda , and S. Jha . 2010. Orthogonal simulated annealing for multiobjective optimization. Computers & Chemical Engineering 34 (10):1618–1631.
Sussman, G. J. 1975. A Computer Model of Skill Acquisition. Elsevier, New York, NY.
Sutton, R. S. , A. G. Barto , and F. Bach . 2018. Reinforcement Learning: An introduction. 2nd ed. MIT Press, Cambridge, MA.
Syswerda, G . 1989. Uniform crossover in genetic algorithms. In 4th International Conference on Genetic Algorithms, edited by J. D. Schaffer . Morgan Kaufmann, San Mateo, CA.
Szegedy, C. , W. Liu , Y. Jia , P. Sermanet , S. Reed , D. Anguelov , D. Erhan , V. Vanhoucke , and A. Rabinovich . 2015. Going deeper with convolutions. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Boston, MA. 1–9.
Tait, R. J. , G. Schaefer , and A. A. Hopgood . 2008. Intensity-based image registration using multiple distributed agents. Knowledge-Based Systems 21(3):256–264.
Takagi, T. , and M. Sugeno . 1985. Fuzzy identification of systems and its applications to modeling and control. *IEEE Trans.* Systems, Man, and Cybernetics 15:116–132.
Tate, A. 1975. Interacting goals and their use. In 4th International Joint Conference on Artificial Intelligence (IJCAI'75). Tbilisi, Georgia. 215–218.
Tate, A. 1977. Generating project networks. In 5th International Joint Conference on Artificial Intelligence (IJCAI). Cambridge, MA. 888–893.

Taunton, J. C. , and D. W. Haspel . 1988. The application of expert system techniques in on-line process control. In Expert Systems in Engineering, edited by D. T. Pham . IFS Publications, Bedford, UK / Springer-Verlag, Berlin, Germany.

To, C. C. , and J. Vohradsky . 2007. A parallel genetic algorithm for single class pattern classification and its application for gene expression profiling in Streptomyces coelicolor. BMC Genomics 8 (49).

Torrey, L. and J. Shavlik . 2009. Transfer learning. In Handbook of Research on Machine Learning Applications and Trends: Algorithms, methods, and techniques, edited by E. Soria , J. Martin , R. Magdalena , M. Martinez , and A. Serrano . 242–264. IGI Global, Hershey, PA.

Tsukimoto, H. 2000. Extracting rules from trained neural networks. IEEE Transactions on Neural Networks 11 (2):377–389.

Turing, A. M. 1950. Computing machinery and intelligence. Mind, LIX (236):433–460.

Ulrich, I. , F. Mondada , and J. D. Nicoud . 1997. Autonomous vacuum cleaner. Robotics and Autonomous Systems 19:233–245.

Ungar, L. H. 1990. A bioreactor benchmark for adaptive network-based process control. In Neural Networks for Control, edited by W. T. Miller , R. S. Sutton and P. J. Werbos . MIT Press, Cambridge, MA.

Valmiki, A. H. , A. A. West , and D. J. Williams . 1991. The evolution of a neural network controller for adhesive dispensing. In IFAC workshop on computer software structures integrating AI/KBS systems in process control. Bergen, Norway. 93–101.

von Seggern, D. 2007. CRC Standard Curves and Surfaces with Mathematics. 2nd ed. CRC Press, Boca Raton, FL.

Wagner, C. , and H. Hagras . 2010. Toward general type-2 fuzzy logic systems based on zSlices. IEEE Transactions on Fuzzy Systems 18 (4):637–660.

Wagner, F. , R. Schmuki , T. Wagner , and P. Wolstenholme . 2006. Modeling Software with Finite State Machines. CRC Press, Boca Raton, FL.

Walker, N. , and J. Fox . 1987. Knowledge-based interpretation of images: A biomedical perspective. Knowledge Engineering Review 2 (4):249–264.

Waser, M. R. 2009. A safe ethical system for intelligent machines. In Biologically Inspired Cognitive Architectures II. North America: AAAI Fall Symposium Series (FS-09-01).

Watson, I , and D. Gardingen . 1999. A distributed case-based reasoning application for engineering sales support. In 16th International Joint Conference on Artificial Intelligence (IJCAI'99). Stockholm, Sweden. 1:600–605.

Welser, J. , J. W. Pitera , and C. Goldberg . 2018. Future computing hardware for AI. In Technical Digest – IEEE International Electron Devices Meeting, IEDM2018.

Wen, W . 2008. A dynamic and automatic traffic light control expert system for solving the road congestion problem. Expert Systems with Applications 34:2370–2381.

Widrow, B. , and M. E. Hoff . 1960. Adaptive switching circuits. In Wescon Conference Record Part 4, 96–104.

Wilkins, D. E. 1983. Representation in a domain-independent planner. In 8th International Joint Conference on Artificial Intelligence (IJCAI'83). Karlsruhe, Germany. 733–740.

Wilkins, D. E. 1984. Domain-independent planning: Representation and plan generation. Artificial Intelligence 22:269–301.

Wilkins, D. E. 1989. Practical Planning: Extending the classical AI planning paradigm. Morgan Kaufmann, San Mateo, CA.

Wilkins, D. E. , and M. desJardins . 2001. A call for knowledge-based planning. AI Magazine 22 (1):99–115.

Willis, M. J. , C. Di Massimo , G. A. Montague , M. T. Tham , and A. J. Morris . 1991. Artificial neural networks in process engineering. IEE Proceedings-D 138 (3):256–266.

Wong, K. , K. Leung , and M. Wong . 2010. Protein structure prediction on a lattice model via multimodal optimization techniques. In GECCO-2010. 155–162.

Wong, T.-T. 2015. Performance evaluation of classification algorithms by *k*-fold and leave-one-out cross validation. Pattern Recognition 48:2839–2846.

Woodcock, N. , N. J. Hallam , and P. D. Picton . 1991a. Fuzzy BOXES as an alternative to neural networks for difficult control problems. In Applications of Artificial Intelligence in Engineering VI, edited by G. Rzevski and R. A. Adey . Computational Mechanics / Elsevier, Southampton, UK. 903–919.

Woodcock, N. , N. J. Hallam , P. D. Picton , and A. A. Hopgood . 1991b. Interpretation of ultrasonic images of weld defects using a hybrid system. In International Conference on Neural Networks and their Applications. Nimes, France.
Wooldridge, M. J. 1997. Agent-based software engineering. IEE Proc. Software Engineering 144 (1):26–37.
Wooldridge, M. J. 1999. Intelligent agents. In Multiagent Systems: A modern approach to distributed artificial intelligence, edited by G. Weiss . MIT Press, Cambridge, MA. 27–77.
Wooldridge, M. J. 2009. An Introduction to Multiagent Systems. 2nd ed. Wiley, Chichester, UK.
Wooldridge, M. J. , and N. R. Jennings . 1994a. Formalising the cooperative problem solving process. In 13th International Workshop on Distributed Artificial Intelligence (IWDAI'94). Lake Quinalt, WA. 403–417.
Wooldridge, M. J. , and N. R. Jennings . 1994b. Towards a theory of cooperative problem solving. In 6th European Conference on Modelling Autonomous Agents in a Multi-Agent World (MAAMAW'94). Odense, Denmark. 15–26.
Wooldridge, M. J. , and N. R. Jennings . 1995. Intelligent agents: Theory and practice. Knowledge Engineering Review 10 (2):115–152.
Wotawa, F. 2000. Debugging VHDL designs using model-based reasoning. Artificial Intelligence in Engineering 14 (4):331–351.
Wu, T. H. , C. C. Chang , and S. H. Chung . 2008. A simulated annealing algorithm for manufacturing cell formation problems. Expert Systems with Applications 34 (3):1609–1617.
Yamada, T. , and R. Nakano . 1992. A genetic algorithm applicable to large-scale job-shop problems. Parallel Problem Solving from Nature 2:281–290.
Yamashita, Y. , H. Komori , E. Aoki , and K. Hashimoto . 2000. Computer aided monitoring of pump efficiency by using ART2 neural networks. Kagaku Kogaku Ronbunshu 26:457–461.
Yang, Q . 1997. Intelligent Planning: A decomposition and abstraction based approach. Springer-Verlag, Berlin, Germany.
Yao, X . 1999. Evolving artificial neural networks. Proceedings of the IEEE 87 (9):1423–1447.
Yella, S. , M. S. Dougherty , and N. K. Gupta . 2006. Artificial intelligence techniques for the automatic interpretation of data from non-destructive testing. Insight 48 (1):10–20.
Yosinski, J. , J. Clune , Y. Bengio , and H. Lipson . 2014. How transferable are features in deep neural networks? In Advances in Neural Information Processing Systems (NIPS'14), edited by Z. Ghahramani , M. Welling , C. Cortes , N. D. Lawrence , and K. Q. Weinberger . Curran Associates, Red Hook, NY. 4:3320–3328.
Zadeh, L. A. 1965. Fuzzy sets. Information and Control 8:338–353.
Zadeh, L. A. 1975. Fuzzy logic and approximate reasoning. Synthese 30:407–428.
Zadeh, L. A. 1983a. Commonsense knowledge representation based on fuzzy logic. IEEE Computer 16 (10):61–65.
Zadeh, L. A. 1983b. The role of fuzzy logic in the management of uncertainty in expert systems. Fuzzy Sets and Systems 11:199–227.
Zadeh, L. A. 1996. Fuzzy logic = computing with words. IEEE Transactions on Fuzzy Systems 4 (2):103–111.
Zhang, J. , J. Zhuang , H. Du , and S. Wang . 2009. Self-organizing genetic algorithm based tuning of PID controllers. Information Sciences 179 (7):1007–1018.
Zhang, J. , W. L. Lo , and H. Chung . 2006. Pseudocoevolutionary genetic algorithms for power electronic circuits optimization. IEEE Trans Systems, Man, and Cybernetics, Part C 36 (4):590–598.
Zhang, X. , X. Zhou , M. Lin , and J. Sun . 2018. ShuffleNet: An extremely efficient convolutional neural network for mobile devices. In IEEE/CVF Conference on Computer Vision and Pattern Recognition. Salt Lake City, UT. 6848–6856.
Zhang, Z. , and M. Simaan . 1987. A rule-based interpretation system for segmentation of seismic images. Pattern Recognition 20 (1):45–53.